Universidad Mariano Gálvez

Curso: Análisis de Sistemas II

Proyecto Final: Análisis, Diseño Arquitectónico Moderno, Aplicación Funcional, UX/UI y Estrategia de Calidad

Integrantes : Joseph Alexander Cruz Alvarado - 9989-23-16312

Emely Victoria Picholá Espinoza - 9989-23-13483

Gabriel Alberto Castellanos Monroy - 9989-23-3898

Eliù Ezequiel De la cruz Santizo - 9989-23-22348

Docente

Fecha 23/10/2026

1. INTRODUCCIÓN Y RESUMEN EJECUTIVO

El presente documento describe el análisis, diseño y especificación del Módulo de Membresías, un componente de software desarrollado como parte del curso de Análisis de Sistemas II en la Universidad Mariano Gálvez de Guatemala. El módulo tiene como finalidad gestionar de forma integral los planes de suscripción, las membresías de los clientes y el procesamiento de pagos dentro de un sistema de gestión deportiva.

1.2 Alcance del software

El Módulo de Membresías abarca exclusivamente las siguientes funcionalidades:

- **Gestión de planes**: creación, modificación y consulta de los planes Básico, Pro y Elite, cada uno con precio, duración y beneficios configurables.
- **Gestión de membresías**: registro, activación, consulta y cancelación de la relación entre un cliente y un plan.
- **Procesamiento de pagos**: cobro a través de una pasarela externa, registro del resultado y notificación al módulo de Reportes.

Quedan fuera del alcance de este módulo: la autenticación de usuarios, la gestión de clientes y la generación de reportes, los cuales son responsabilidad de módulos externos con los que este sistema se integra mediante APIs REST.

1.3 Síntesis de los pasos metodológicos aplicados

El desarrollo del módulo siguió una metodología estructurada de diez pasos, combinando estándares internacionales de ingeniería de software con prácticas ágiles:

1. **Identificación del dominio**: delimitación del alcance al Módulo de Membresías y definición de los módulos externos (Clientes, Autenticación, Reportes).
2. **Especificación de requerimientos (ISO/IEC/IEEE 29148:2018)**: documentación de 32 requerimientos funcionales (REQ-FN-001 al REQ-FN-032) y 16 no funcionales (REQ-NF-001 al REQ-NF-016), con identificadores únicos, origen y prioridad MoSCoW.
3. **Modelado UML 2.5.1**: elaboración de diagramas de clases (análisis y diseño), diagramas de secuencia (registrar membresía, procesar pago), diagrama de actividades con swimlanes y diagrama de paquetes.
4. **Diseño de componentes (IEEE 1016-2009)**: fichas de diseño detalladas para los seis componentes del módulo: Middleware JWT, Planes, Membresías, Pagos, Adaptadores de integración y Caché del catálogo (Redis).
5. **Diseño de arquitectura C4**: modelado en niveles de contexto, contenedores y componentes para visualizar la estructura del sistema y sus integraciones.
6. **Diseño de base de datos**: esquema relacional en SQL Server con las entidades Planes, Membresias y Pagos, incluyendo restricciones de integridad y una vista de membresías activas.
7. **Especificación de la API REST (OpenAPI 3.0.3)**: documentación de los diez endpoints del módulo con sus códigos de respuesta, esquemas de datos y autenticación JWT Bearer.
8. **Gestión ágil del producto**: elaboración del Product Backlog con 14 historias de usuario y 8 reglas de negocio, estimadas en story points bajo el criterio INVEST, con Definición de Listo (DoR) y Definición de Terminado (DoD).
9. **Trazabilidad**: matriz de trazabilidad que vincula requerimientos, componentes de diseño y elementos UML, garantizando cobertura completa del alcance definido.
10. **Verificación y validación**: revisión cruzada de nombres, códigos y contratos entre el SRS, el SDD, los diagramas UML, el esquema de base de datos y la especificación OpenAPI.

SRS — Módulo de Membresías:

1. Introducción

Este documento especifica los requerimientos del Módulo de Membresías bajo la norma ISO/IEC/IEEE 29148:2018, y sirve de base para el diseño (UML, fichas IEEE 1016) y el backlog ágil.

1.1 Propósito

Definir de forma verificable qué debe hacer el módulo y con qué calidad, para que el equipo de desarrollo, el Product Owner y quien evalúe el proyecto compartan una sola fuente de verdad. Destinatarios: analista, desarrolladores, QA, docente.

1.2 Alcance

El producto se llama **API de Modulos de Membresías** y expone servicios REST para tres capacidades:

- **Catálogo de planes:** administrar y consultar los planes Básico, Pro y Elite.
- **Gestión de membresías:** registrar, consultar y cancelar membresías, y calcular su estado (Activo, Por vencer, Vencido).
- **Procesamiento de pagos:** cobrar la membresía con tarjeta mediante la pasarela externa y guardar cada transacción.

Fuera de alcance: autenticación y emisión del JWT, ficha completa del cliente y generación de reportes. Son módulos externos con los que esta API solo se integra.

1.3 Definiciones y acrónimos

Término	Definición
API	Interfaz de programación de aplicaciones; aquí, servicios REST sobre HTTPS con cuerpo JSON
JWT	JSON Web Token; credencial firmada emitida por el Módulo de Autenticación

Plan	Oferta comercial con nombre, precio, duración y beneficios: Básico, Pro o Elite
Membresía	Contrato vigente de un cliente con un plan, con fecha de inicio y fecha de fin
Estado visual	Activo, Por vencer (7 días o menos para fecha_fin) o Vencido; se calcula, no se almacena
Pasarela de Pago	Sistema externo que autoriza o rechaza el cobro con tarjeta
SRS	Software Requirements Specification, este documento
REQ-FN / REQ-NF	Identificadores de requerimiento funcional / no funcional
HU / RN	Historia de usuario / regla de negocio del backlog

2. Descripción general

La API de Membresías es un servicio independiente, consumido por la app móvil del gimnasio, que se apoya en cuatro sistemas externos y en su propia base de datos.

2.1 Perspectiva del producto

La API se desarrolla como módulo autónomo (monolito modular). Persiste planes, membresías y pagos en SQL Server y guarda el catálogo de planes en caché Redis (patrón Cache-Aside). Depende de:

- **Módulo de Autenticación:** emite y valida el JWT.
- **Módulo de Clientes:** confirma que el cliente existe y está Activo.
- **Pasarela de Pago:** autoriza o rechaza el cobro.
- **Módulo de Reportes:** recibe datos de ingresos y vencimientos.

2.2 Funciones principales

1. Consultar, crear, editar y desactivar planes (Básico, Pro, Elite).
2. Registrar, consultar y cancelar membresías.
3. Calcular el estado visual (Activo, Por vencer, Vencido) de cada membresía.
4. Procesar pagos con tarjeta y guardar cada intento (aprobado o rechazado).
5. Proteger todos los servicios con JWT y validar al cliente antes de vender.
6. Enviar datos de ingresos y vencimientos al Módulo de Reportes.

2.3 Clases de usuario

Rol	Qué hace en el módulo	Servicios que usa
Cliente	Consulta planes y el estado de su membresía	Catálogo, estado de membresía
Recepcionista	Registra y cancela membresías y cobra	Membresías, pagos
Administrador	Gestiona planes y consulta el historial de pagos	Planes, historial de pagos
Módulos externos	Autenticación, Clientes, Pasarela, Reportes	Interfaces de la sección 5

2.4 Restricciones

- El catálogo contiene tres planes (Básico, Pro y Elite) y solo los planes activos pueden venderse (RN-04).
- Base de datos SQL Server; caché Redis; contrato REST documentado en OpenAPI 3.0.3.
- Formato de error único: { success: false, error: { code, message } }.
- Moneda por defecto: GTQ; el monto de un pago es mayor a 0.

2.5 Supuestos y dependencias

- El Módulo de Autenticación está disponible y entrega JWT con el rol del usuario.
- El Módulo de Clientes responde de forma consultable por identificador de cliente.
- La pasarela de pago es un servicio externo; sus fallas no deben corromper el estado de una membresía.
- Si un supuesto cambia (por ejemplo, otro proveedor de pagos), cambian los requerimientos de la sección 5.

3. Requerimientos funcionales

ID	Requerimiento	Origen	Prioridad
REQ-FN-001	El sistema debe devolver el nombre, precio, duración y beneficios de los planes con activo = 1 cuando reciba GET /planes.	HU-01, RN-04	Must
REQ-FN-002	El sistema debería crear un plan cuando el Administrador envíe POST /planes con nombre Básico, Pro o Elite que aún no exista.	HU-02	Should
REQ-FN-003	El sistema debería actualizar el precio y los beneficios de un plan cuando el Administrador envíe PUT /planes/{id} y el plan exista.	HU-02	Should
REQ-FN-004	El sistema debería rechazar con 400 la creación o edición de un plan cuando el precio sea menor o igual a 0 o el nombre no sea Básico, Pro o Elite.	HU-02	Should
REQ-FN-005	El sistema podrá desactivar un plan (activo = 0) sin eliminarlo cuando el Administrador envíe PATCH /planes/{id}, conservando las membresías que ya lo usan.	HU-03, RN-04	Could

REQ-FN-006	El sistema debería invalidar el catálogo almacenado en caché cuando se cree, edite o desactive un plan.	HU-02, HU-03	Should

3.2 Gestión de membresías (EPIC-02)

ID	Requerimiento	Origen	Prioridad
REQ-FN-007	El sistema debe registrar una membresía con estado Activa cuando reciba POST /membresias con un cliente válido, un plan activo y fechas válidas.	HU-04	Must
REQ-FN-008	El sistema debe rechazar con 400 el registro de una membresía cuando fecha_fin no sea posterior a fecha_inicio.	RN-03	Must
REQ-FN-009	El sistema debe rechazar el registro de una membresía con 404 cuando el plan no exista y con 422 cuando el plan esté inactivo.	RN-04	Must
REQ-FN-010	El sistema debe devolver las membresías de un cliente, de la más reciente a la más antigua, cuando reciba GET /membresias con id_cliente.	HU-05	Must
REQ-FN-011	El sistema debe devolver los datos de una membresía, incluido su estado visual, cuando reciba GET /membresias/{id}, y responder 404 cuando no exista.	HU-06	Must
REQ-FN-012	El sistema debe calcular el estado visual en cada consulta: Vencido si la membresía está cancelada o su fecha_fin ya pasó, Por vencer si faltan 7 días o menos, y Activo en cualquier otro caso.	HU-06, HU-07, RN-05	Must

REQ-FN-013	El sistema podrá cancelar una membresía cuando el Recepcionista envíe PATCH /membresias/{id} con estado Cancelada.	HU-08	Could
REQ-FN-014	El sistema podrá rechazar con 409 la cancelación de una membresía cuando ya esté Cancelada o Vencida.	HU-08	Could

3.3 Procesamiento de pagos

ID	Requerimiento	Origen	Prioridad
REQ-FN-015	El sistema debe procesar el pago de una membresía cuando reciba POST /payments/process con membresía existente, monto y moneda válidos.	HU-09, RN-06, RN-07	Must
REQ-FN-016	El sistema debe rechazar con 400 un pago cuando el monto sea menor o igual a 0 o la moneda no sea un código ISO 4217.	RN-06	Must
REQ-FN-017	El sistema debe rechazar con 404 un pago cuando la membresía indicada no exista.	RN-07	Must
REQ-FN-018	El sistema debe enviar la solicitud de cobro a la Pasarela de Pago y responder con la decisión (APPROVED o DECLINED) y el transaction_id cuando el pago sea válido.	HU-09	Must
REQ-FN-019	El sistema debe registrar cada intento de pago, aprobado o rechazado, con monto, moneda, estado, transaction_id y fecha.	HU-10	Must

REQ-FN-020	El sistema debe asignar a cada pago registrado un transaction_id único.	HU-10	Must
REQ-FN-021	El sistema debe responder 504 y no registrar el pago como APPROVED cuando la Pasarela de Pago no responda dentro del tiempo límite.	HU-09	Must
REQ-FN-022	El sistema debería devolver los pagos de una membresía cuando el Administrador envíe GET /membresias/{id}/pagos.	HU-11	Should

3.4 Integración con otros módulos

ID	Requerimiento	Origen	Prioridad
REQ-FN-023	El sistema debe validar la firma y la vigencia del JWT cuando reciba una petición a un endpoint protegido.	HU-12, RN-01	Must
REQ-FN-024	El sistema debe responder 401 cuando falte el encabezado Authorization: Bearer <token> o el token sea inválido o esté vencido.	RN-01	Must
REQ-FN-025	El sistema debe responder 403 cuando el rol contenido en el token no tenga permiso sobre el endpoint solicitado.	HU-12, HU-02, HU-11	Must
REQ-FN-026	El sistema debe consultar al Módulo de Clientes si el cliente existe y está Activo antes de registrar una membresía.	HU-13, RN-02	Must

REQ-FN-027	El sistema debe rechazar el registro de una membresía con 404 cuando el cliente no exista y con 422 cuando el cliente no esté Activo.	HU-13, RN-02	Must
REQ-FN-028	El sistema debe responder 503 y no crear la membresía cuando el Módulo de Clientes no esté disponible.	HU-13	Must
REQ-FN-029	El sistema debería enviar al Módulo de Reportes los datos de ingresos y vencimientos cuando registre un pago APPROVED.	HU-14	Should
REQ-FN-030	El sistema debería completar el pago y dejar el envío pendiente de reintento cuando el Módulo de Reportes no responda.	HU-14	Should

3.5 Formato de respuestas

ID	Requerimiento	Origen	Prioridad
REQ-FN-031	El sistema debe responder todo error con el formato { success: false, error: { code, message } }.	RN-08	Must
REQ-FN-032	El sistema debe responder toda consulta exitosa con el formato { success: true, data }.	RN-08	Must

4. Requerimientos no funcionales

4.1 Seguridad

ID	Requerimiento	Verificación
REQ-NF-001	El sistema debe aceptar peticiones únicamente por HTTPS con TLS 1.2 o superior.	Una petición HTTP sin cifrado es rechazada o redirigida
REQ-NF-002	El sistema debe rechazar con 401 todo JWT con firma alterada o con algoritmo none.	Prueba con un token manipulado devuelve 401
REQ-NF-003	El sistema no debe almacenar número de tarjeta ni CVV; solo debe guardar el transaction_id de la pasarela.	Revisión de schema.sql: ninguna columna con datos de tarjeta
REQ-NF-004	El sistema debe ejecutar todo acceso a SQL Server mediante consultas parametrizadas.	Revisión de código y prueba de inyección SQL sin efecto
REQ-NF-005	El sistema no debe incluir trazas de pila ni sentencias SQL en el mensaje de error que devuelve al cliente.	Provocar un error interno y revisar el cuerpo de la respuesta

4.2 Rendimiento

ID	Requerimiento	Verificación
REQ-NF-006	El sistema debe responder GET /planes en 200 ms o menos (percentil 95) con 100 usuarios concurrentes cuando el catálogo esté en caché.	Prueba de carga
REQ-NF-007	El sistema debe responder las consultas de membresías en 500 ms o menos (percentil 95) con 100 usuarios concurrentes.	Prueba de carga
REQ-NF-008	El sistema debe esperar la respuesta de la Pasarela de Pago un máximo de 10 segundos antes de responder 504.	Simular una pasarela lenta y medir el tiempo de respuesta
REQ-NF-009	El sistema debe expirar el catálogo en caché a más tardar 10 minutos después de cargarlo.	Consultar el TTL de la clave en Redis

4.3 Disponibilidad y tolerancia a fallos

ID	Requerimiento	Verificación
REQ-NF-010	El sistema debe estar disponible al menos 99 % del tiempo en cada mes.	Monitoreo mensual de disponibilidad
REQ-NF-011	El sistema debe seguir respondiendo GET /planes desde SQL Server cuando Redis no esté disponible.	Apagar Redis y verificar que la respuesta sigue siendo 200

4.4 Usabilidad e interoperabilidad

ID	Requerimiento	Verificación
REQ-NF-012	El sistema debe publicar un contrato OpenAPI 3.0.3 que valide sin errores y describa todos los endpoints implementados.	Validar el YAML en Swagger Editor
REQ-NF-013	El sistema debe devolver en cada error un code constante y un message en español que indique qué dato corregir.	Revisión de los mensajes de error en pruebas de API

4.5 Mantenibilidad e integridad de datos

ID	Requerimiento	Verificación
REQ-NF-014	El sistema debe separar el código en capas Controllers, Services y Repositories, sin acceso directo de los Controllers a la base de datos.	Revisión del diagrama de paquetes y de las dependencias del código
REQ-NF-015	El sistema debe contar con pruebas unitarias que cubran al menos 70 % de las líneas de la capa Services.	Reporte de cobertura
REQ-NF-016	El sistema debe imponer en la base de datos las claves foráneas y las restricciones fecha_fin > fecha_inicio y monto > 0.	Intentar insertar datos que violen cada restricción y verificar el rechazo

Diagramación UML:

Diagrama de Clases de Análisis



2. Clases de diseño

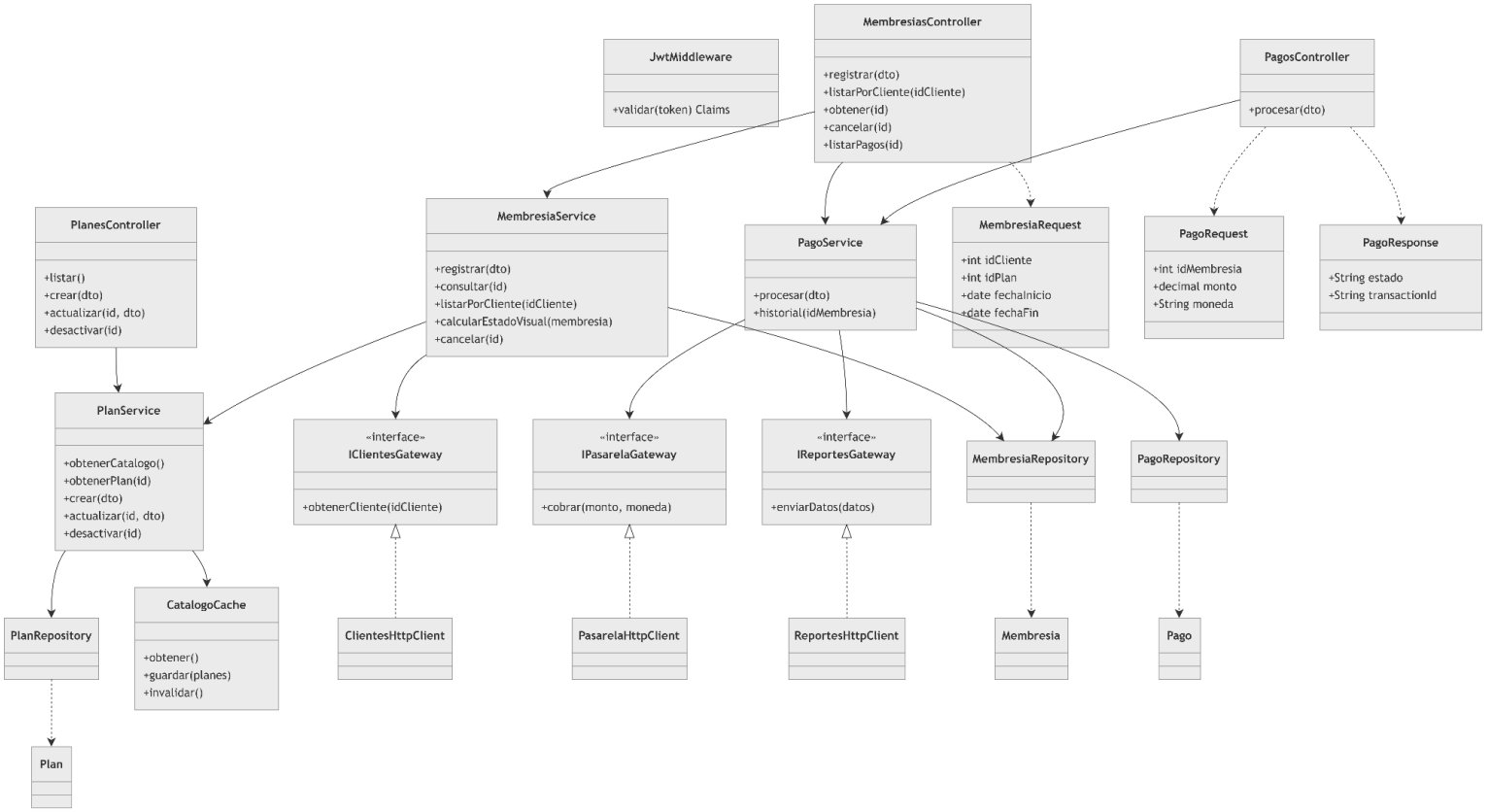
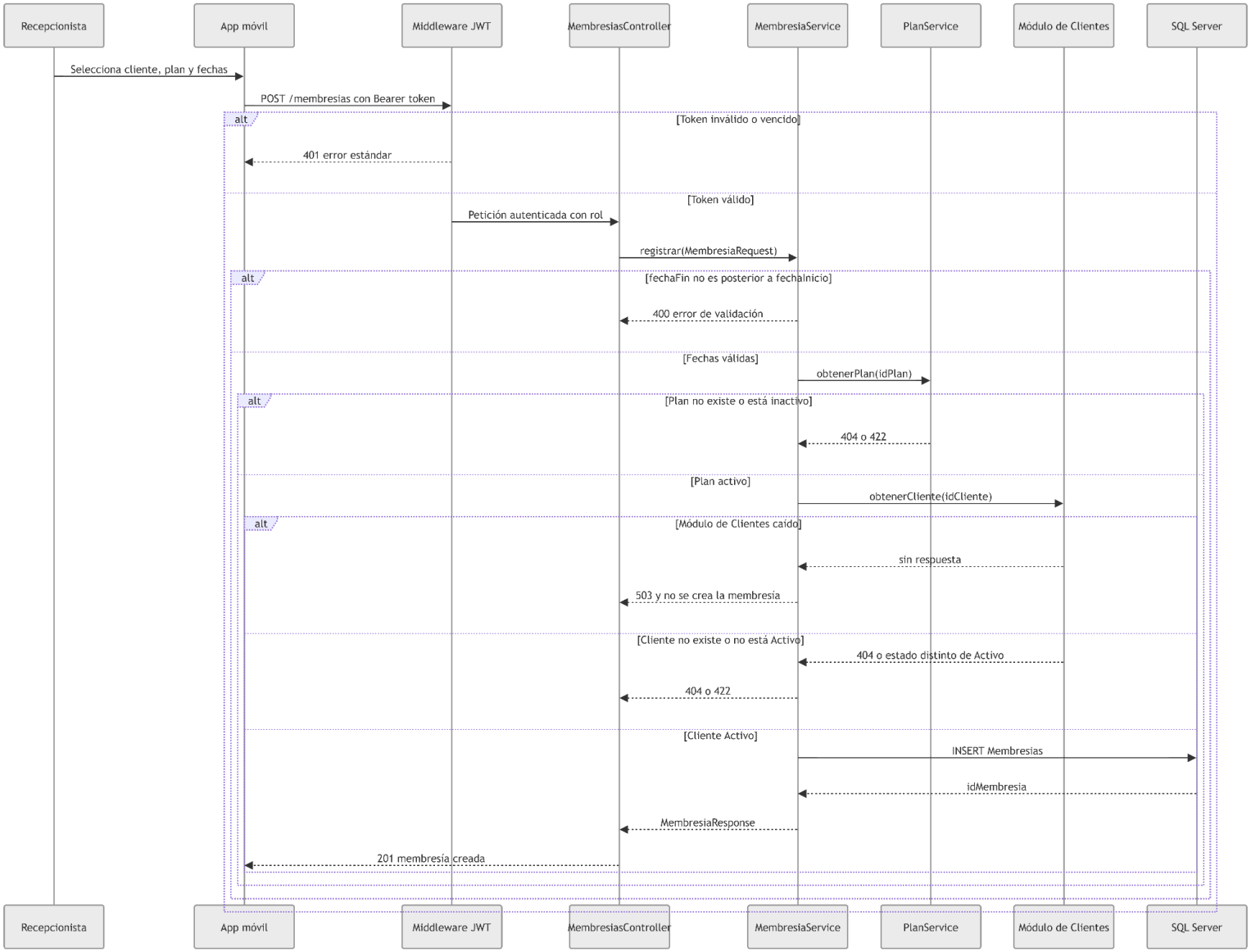
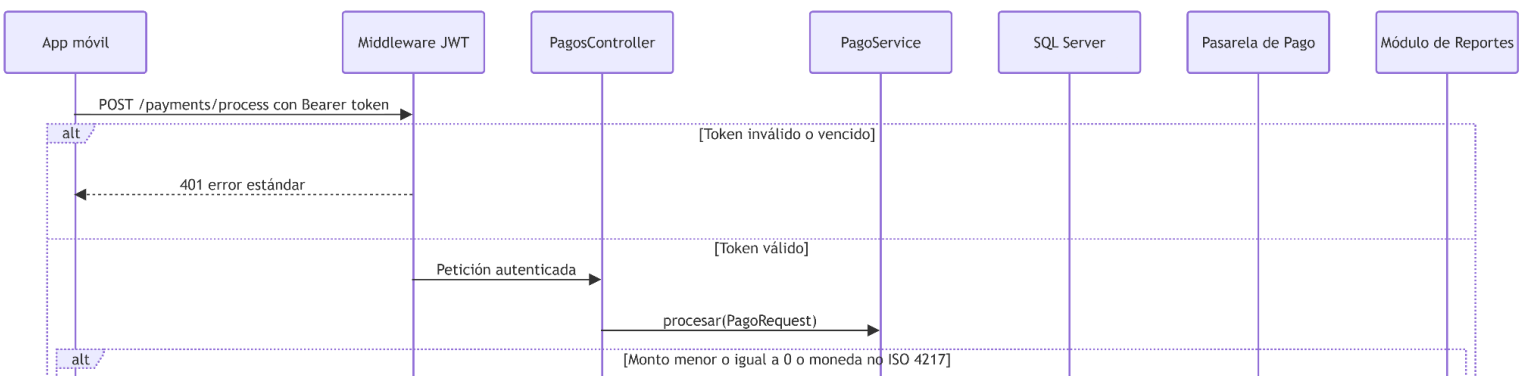


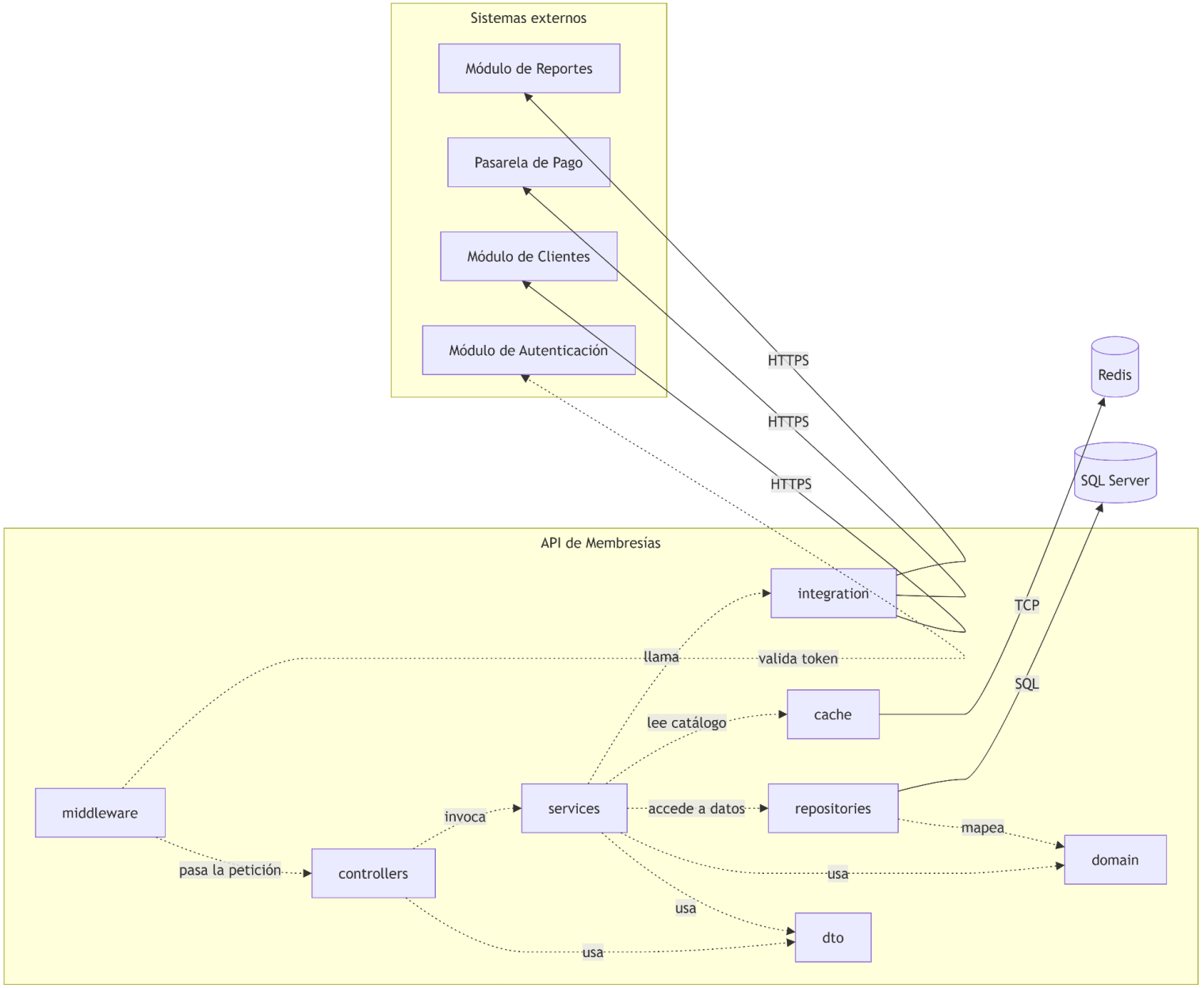
Diagrama de Secuencia para las interacciones críticas.



4. Secuencia: procesar un pago



6. Paquetes (modularización)



3.Fichas de diseño de componentes (IEEE 1016-2009):

Modulo de Membresias se organiza en componentes funcionales que interactúan entre sí mediante interfaces definidas. La interfaz de usuario solicita operaciones a la lógica de negocio, que valida las reglas correspondientes y accede a SQL Server mediante la capa de datos. Para el procesamiento de pagos se contempla, además, la comunicación con una pasarela externa.

Los componentes principales son:

1. Autenticación e inicio de sesión.
2. Panel principal o Dashboard.
3. Gestión de clientes.
4. Gestión de planes.
5. Gestión de membresías.
6. Gestión de pagos y pasarela de pago.
7. Reportes y consultas.

. Fichas de diseño de componentes

3.1. Componente de autenticación e inicio de sesión

Identificador: SDD-CMP-001

Nombre: Módulo de autenticación.

Propósito: Permitir el acceso seguro al sistema mediante la validación de las credenciales del usuario y la identificación de sus permisos.

Responsabilidades:

- Capturar el usuario o correo electrónico y la contraseña.
- Validar que las credenciales estén completas.
- Verificar las credenciales mediante el servicio de autenticación.
- Iniciar la sesión cuando las credenciales sean válidas.
- Mostrar mensajes de error cuando la autenticación falle.
- Restringir el acceso a las funciones según los permisos asignados.

Atributos principales:

Atributo	Tipo	Descripción
IdUsuario	INT	Identificador del usuario.
NombreUsuario	VARCHAR	Nombre de usuario registrado.
Correo	VARCHAR	Correo asociado a la cuenta, si corresponde.
HashContrasena	VARCHAR	Representación segura de la contraseña almacenada.
Rol	VARCHAR	Rol asignado, por ejemplo, Administrador.
Estado	BIT	Indica si la cuenta está habilitada.

Interfaces:

- IniciarSesion(usuario, contrasena): solicita la autenticación y devuelve el resultado.
- CerrarSesion(): finaliza la sesión activa.
- ValidarPermisos(rol, recurso): verifica si el usuario puede acceder a una funcionalidad.

Dependencias:

- Formulario de inicio de sesión.
- Servicio de autenticación.
- Base de datos SQL Server, tabla Usuarios.
- Módulo de autorización.

Manejo de excepciones:

- Credenciales incorrectas: mostrar un mensaje de acceso denegado.
- Campos vacíos: solicitar que se completen los datos.
- Error de conexión con la base de datos: informar que el servicio no está disponible.
- Cuenta deshabilitada: impedir el acceso.
- Error inesperado: registrar el incidente y mostrar un mensaje controlado sin exponer información sensible.

Consideraciones de diseño: Las contraseñas no deben almacenarse en texto plano. Se utilizará un algoritmo de hash de contraseñas con salt. Si se emplea JWT para acceder a servicios externos, el token deberá validarse y administrarse de forma segura.

3.2. Componente del panel principal (Dashboard)

Nombre: Panel principal de FitControl.

Propósito: Presentar un resumen del estado del gimnasio mediante indicadores, actividad reciente y accesos a los módulos principales.

Responsabilidades:

- Mostrar la cantidad de clientes activos.
- Presentar los ingresos registrados durante el período seleccionado.
- Mostrar las membresías próximas a vencer.
- Informar sobre nuevas altas de membresías.
- Mostrar las actividades recientes del sistema.
- Permitir la navegación hacia clientes, planes, membresías y pagos.

Atributos principales:

Atributo	Tipo	Descripción
TotalClientesActivos	**INT**	**Cantidad de clientes con membresía activa.**
IngresosPeriodo	**DECIMAL(10,2)**	**Total de ingresos del período consultado.**
MembresiasPorVencer	**INT**	**Membresías próximas a finalizar.**
NuevasMembresias	**INT**	**Membresías registradas en el período.**
ActividadReciente	**LIST**	**Operaciones recientes que se muestran al usuario.**

Interfaces:

- ObtenerResumen(): recupera los indicadores del panel.
- ObtenerActividadReciente(): consulta las últimas operaciones registradas.
- NavegarModulo(nombreModulo): abre el módulo seleccionado.

Dependencias:

- Módulos de clientes, membresías y pagos.
- Base de datos SQL Server.
- Servicio de consultas y agregación de indicadores.

Manejo de excepciones:

- Error al consultar indicadores: mostrar un aviso y permitir reintentar.
- Ausencia de registros: mostrar valores en cero o un mensaje de información.
- Error de conexión: mantener la interfaz disponible y notificar que los datos no pudieron actualizarse.
- Datos desactualizados: permitir actualizar el panel mediante una nueva consulta.

Consideraciones de diseño: Los indicadores deben calcularse a partir de los registros existentes en la base de datos. Los ingresos deberán considerar únicamente los pagos que se encuentren confirmados según las reglas del sistema.

3.3. Componente de gestión de clientes

Nombre: Módulo de clientes.

Propósito: Administrar la información personal y el estado de los clientes registrados en el gimnasio.

Responsabilidades:

- Registrar nuevos clientes.
- Consultar y listar clientes.
- Buscar clientes por nombre u otros criterios.
- Modificar información personal.
- Consultar el estado de las membresías asociadas.
- Permitir el acceso a la información de cada cliente.

Atributos principales:

Atributo	Tipo	Descripción
IdCliente	**INT**	**Identificador único del cliente.**
Nombre	**VARCHAR**	**Nombre del cliente.**
Apellido	**VARCHAR**	**Apellido del cliente.**
Telefono	**VARCHAR**	**Número de contacto.**
Correo	**VARCHAR**	**Correo electrónico.**
FechaRegistro	**DATE**	**Fecha de registro en el gimnasio.**
Estado	**VARCHAR**	**Estado del cliente según las reglas del sistema.**

Interfaces:

- RegistrarCliente(cliente): guarda un nuevo cliente.
- ListarClientes(filtros): recupera clientes según los filtros aplicados.
- BuscarCliente(idCliente): obtiene la información de un cliente.
- ActualizarCliente(cliente): modifica sus datos.
- ConsultarMembresias(idCliente): obtiene las membresías asociadas al cliente.

Dependencias:

- Formulario de clientes.

- Componente de membresías.
- Capa de lógica de negocio.
- Base de datos SQL Server, tabla Clientes.

Manejo de excepciones:

- Campos obligatorios vacíos: impedir el registro.
- Correo con formato inválido: solicitar su corrección.
- Cliente duplicado según los criterios de unicidad definidos: rechazar el registro.
- Cliente inexistente: mostrar un mensaje informativo.
- Error al guardar o actualizar: informar el resultado y evitar mostrar una operación como exitosa si no se confirmó.

Consideraciones de diseño: Las operaciones deberán validar los datos antes de enviarlos a la base de datos y utilizar consultas parametrizadas para reducir riesgos de inyección SQL.

3.4. Componente de gestión de planes

Nombre: Módulo de planes.

Propósito: Administrar los planes comerciales que el gimnasio ofrece a sus clientes.

Responsabilidades:

- Registrar nuevos planes.
- Consultar y listar los planes disponibles.
- Modificar precios, duración y descripción.
- Activar o desactivar planes.
- Proporcionar información a los módulos de membresías y pagos.

Atributos principales:

Atributo	Tipo	Descripción
IdPlan	**INT**	**Identificador único del plan.**
NombrePlan	**VARCHAR**	**Nombre comercial del plan.**
Descripcion	**VARCHAR**	**Detalle de los beneficios ofrecidos.**
Precio	**DECIMAL(10,2)**	**Precio del plan.**
DuracionDias	**INT**	**Duración del plan en días.**
Estado	**BIT**	**Indica si el plan está disponible.**

Interfaces:

- RegistrarPlan(plan): crea un plan.

- ListarPlanes(): obtiene los planes registrados.
- ActualizarPlan(plan): modifica los datos del plan.
- ConsultarPlan(idPlan): recupera los datos de un plan.
- CambiarEstadoPlan(idPlan, estado): habilita o deshabilita un plan.

Dependencias:

- Formulario de planes.
- Módulo de membresías.
- Módulo de pagos.
- Base de datos SQL Server, tabla Planes.

Manejo de excepciones:

- Precio negativo o duración no válida: rechazar el registro.
- Plan inexistente: informar que no se encontró.
- Plan asociado a membresías existentes: impedir su eliminación física cuando afecte la integridad referencial; permitir desactivarlo según las reglas del negocio.
- Error de persistencia: informar que la operación no se completó.

Consideraciones de diseño: Los precios y las duraciones deberán validarse antes de registrar un plan. Los cambios en los planes no deberán modificar retroactivamente los pagos ya realizados.

3.5. Componente de gestión de membresías

Nombre: Módulo de membresías.

Propósito: Controlar la contratación, vigencia y estado de las membresías de los clientes del gimnasio.

Responsabilidades:

- Registrar una membresía para un cliente y un plan.
- Calcular o validar las fechas de inicio y vencimiento.
- Consultar las membresías registradas.
- Actualizar los datos permitidos de una membresía.
- Identificar membresías activas, próximas a vencer, vencidas o canceladas.
- Relacionar la membresía con los pagos correspondientes.

Atributos principales:

Atributo	Tipo	Descripción
IdMembresia	**INT**	**Identificador único de la membresía.**
IdCliente	**INT**	**Referencia al cliente titular.**
IdPlan	**INT**	**Referencia al plan contratado.**

FechaInicio	DATE	Fecha de inicio de la membresía.
FechaFin	DATE	Fecha de vencimiento.
Estado	VARCHAR(20)	Activa, Vencida o Cancelada.

Interfaces:

- RegistrarMembresia(membresia): registra una membresía.
- ListarMembresias(filtros): consulta membresías por cliente, estado o fecha.
- BuscarMembresia(idMembresia): obtiene una membresía específica.
- ActualizarMembresia(membresia): modifica los campos permitidos.
- CalcularFechaFin(fechaInicio, duracionDias): determina la fecha de finalización según la duración del plan.
- ActualizarEstadoMembresia(idMembresia): verifica y actualiza el estado según las fechas y las reglas del negocio.

Dependencias:

- Módulo de clientes.
- Módulo de planes.
- Módulo de pagos.
- Capa de acceso a datos, por ejemplo, MembresiaDAL.
- Capa de lógica de negocio, por ejemplo, MembresiaBLL.
- Base de datos SQL Server, tabla Membresias.

Manejo de excepciones:

- Cliente inexistente: impedir la asignación de la membresía.
- Plan inexistente o desactivado: impedir la contratación.
- Fecha final anterior o igual a la fecha inicial: rechazar los datos.
- Membresía inexistente: informar que no se encontró.
- Error de base de datos: cancelar la operación y conservar la consistencia de los registros.
- Membresía vencida: actualizar o presentar el estado correspondiente sin confundirlo con un pago pendiente.

Consideraciones de diseño: La vigencia de la membresía y el estado del pago son conceptos distintos. Una membresía no deberá considerarse pagada únicamente por haber sido creada. Si el negocio exige confirmar el pago antes de activar la membresía, ambas operaciones deberán coordinarse para evitar activaciones sin cobro confirmado.

3.6. Componente de gestión de pagos y pasarela de pago

Nombre: Módulo de pagos.

Propósito: Registrar los pagos de las membresías y comunicarse con una pasarela de pago externa para procesar transacciones electrónicas.

Responsabilidades:

- Mostrar el monto que debe pagar el cliente.
- Asociar el pago con el cliente y la membresía correspondiente.
- Enviar la solicitud de pago a la pasarela externa.
- Procesar la respuesta de aprobación o rechazo.
- Registrar el identificador de la transacción.
- Mantener el historial y el estado de los pagos.
- Notificar al módulo de membresías cuando un pago quede confirmado.

Atributos principales:

Atributo	Tipo	Descripción
IdPago	**INT**	**Identificador interno del pago.**
IdMembresia	**INT**	**Membresía a la que corresponde el pago.**
Monto	**DECIMAL(10,2)**	**Importe que se pretende cobrar.**
Moneda	**VARCHAR(3)**	**Moneda de la transacción, por ejemplo, GTQ.**
FechaPago	**DATETIME**	**Fecha y hora del pago.**
Estado	**VARCHAR(20)**	**Pendiente, Aprobado o Rechazado.**
TransactionId	**VARCHAR**	**Identificador de la transacción externa, cuando exista.**
CardToken	**VARCHAR**	**Token de pago proporcionado para la transacción, si corresponde.**

Interfaces:

- ProcesarPago(idMembresia, monto, moneda, cardToken): solicita el procesamiento de un pago.
- ConsultarPago(idPago): recupera la información de un pago.
- ConsultarEstadoTransaccion(transactionId): consulta el estado de una transacción.
- RegistrarResultadoPago(resultado): guarda el resultado confirmado.
- NotificarPagoConfirmado(idMembresia): comunica la confirmación al módulo de membresías.

Dependencias:

- Módulo de membresías.
- Base de datos SQL Server, tabla Pagos.
- Servicio de integración con la pasarela de pago externa.
- Servicio de autenticación y autorización.
- Conexión segura a Internet.

Manejo de excepciones:

- Monto inválido o negativo: rechazar la solicitud antes de enviarla.
- Membresía inexistente: impedir el cobro.
- Pago rechazado: registrar el resultado como rechazado cuando exista una respuesta confirmada.
- Tiempo de espera agotado: marcar el resultado como pendiente de verificación cuando no se sepa si la pasarela procesó el cobro.
- Error de conexión con la pasarela: informar que no fue posible confirmar la operación.
- Transacción duplicada: verificar la operación previa y evitar cobros repetidos mediante un mecanismo de idempotencia cuando el proveedor lo permita.
- Error al guardar el resultado: no informar que el pago quedó registrado correctamente; reconciliar la operación con la pasarela antes de reintentar un cobro.

Consideraciones de diseño: La integración se realizará mediante la API documentada del proveedor de pagos que se seleccione. El uso de cardToken representa el envío de un token de pago, no de los datos completos de la tarjeta. No se deberán almacenar números completos de tarjetas ni códigos de seguridad. Las credenciales del proveedor y los tokens deberán protegerse.

El endpoint POST /payments/process de la API FitControl podrá recibir amount, currency y cardToken, exigir autenticación Bearer cuando corresponda y devolver el identificador de transacción junto con su estado. El resultado deberá reflejar la respuesta real de la pasarela, no una aprobación simulada.

3.7. Componente de reportes y consultas

Propósito: Proporcionar información consolidada sobre clientes, membresías, ingresos y pagos para apoyar las actividades administrativas del gimnasio.

Responsabilidades:

- Consultar clientes registrados.
- Generar listados de membresías activas, vencidas y próximas a vencer.
- Consultar los pagos confirmados por período.
- Calcular indicadores e ingresos a partir de los registros almacenados.
- Presentar los resultados en pantalla y permitir su exportación cuando esa función esté implementada.

Atributos principales:

Atributo	Tipo	Descripción
TipoReporte	**VARCHAR**	**Tipo de reporte solicitado.**
FechaInicio	**DATE**	**Inicio del período de consulta.**
FechaFin	**DATE**	**Fin del período de consulta.**

TotalRegistros	INT	Número de registros obtenidos.
TotalIngresos	DECIMAL(10,2)	Total calculado de pagos confirmados.
FechaGeneracion	DATETIME	Momento en que se genera el reporte.

Interfaces:

- GenerarReporte(tipo, fechaInicio, fechaFin): genera un reporte.
- ConsultarIngresos(fechaInicio, fechaFin): calcula los ingresos confirmados.
- ConsultarMembresiasPorEstado(estado): obtiene las membresías según su estado.
- ExportarReporte(formato): exporta los resultados a un formato compatible, si está habilitado.

Dependencias:

- Módulos de clientes, membresías y pagos.
- Base de datos SQL Server.
- Componente de presentación de reportes.
- Biblioteca de exportación, si corresponde.

Manejo de excepciones:

- Rango de fechas inválido: solicitar un rango correcto.
- Ausencia de datos: mostrar un reporte vacío o un mensaje informativo.
- Error de consulta: informar que no se pudo generar el reporte.
- Error de exportación: notificar el problema y permitir intentar nuevamente.

Consideraciones de diseño: Los reportes deberán consultar información consistente y respetar los permisos del usuario. Los ingresos se calcularán a partir de pagos confirmados, excluyendo operaciones rechazadas o pendientes.

4. Interfaces y dependencias entre componentes

La interacción entre los módulos deberá seguir el orden lógico del negocio:

1. El usuario inicia sesión y obtiene acceso según sus permisos.
2. El panel principal consulta los indicadores necesarios.
3. El módulo de clientes proporciona la información del titular de la membresía.
4. El módulo de planes proporciona el precio y la duración contratados.
5. El módulo de membresías registra la contratación y controla su vigencia.
6. El módulo de pagos solicita la transacción a la pasarela y registra el resultado.
7. Los módulos de dashboard y reportes consultan los datos almacenados para mostrar información actualizada.

Las dependencias deberán mantenerse controladas. Por ejemplo, el módulo de reportes consultará los datos de pagos y membresías sin asumir la responsabilidad de modificarlos directamente.

5. Criterios generales de manejo de excepciones

Todos los componentes deberán cumplir con los siguientes criterios:

- Validar los datos antes de ejecutar operaciones de persistencia.
- Utilizar consultas parametrizadas para interactuar con SQL Server.
- Gestionar las excepciones sin mostrar detalles internos, contraseñas ni credenciales.
- Registrar los errores técnicos relevantes para facilitar el diagnóstico.
- Informar al usuario mediante mensajes comprensibles.
- Utilizar transacciones de base de datos cuando una operación implique varios cambios que deban confirmarse conjuntamente.
- Evitar que un error en un componente produzca registros inconsistentes en otros módulos.
- Distinguir entre errores de validación, errores de acceso, fallos de infraestructura y resultados de negocio, como un pago rechazado.

Gestión Ágil del Proyecto (Scrum/Kanban)

Roles y responsabilidades

1. Analista de sistemas

El Analista de Sistemas será responsable de comprender las necesidades del gimnasio y transformarlas en requerimientos funcionales y no funcionales que puedan ser implementados y probados.

Sus responsabilidades serán:

- Recopilar y documentar los requerimientos de FitControl.
- Analizar los procesos de clientes, planes, membresías, pagos y reportes.
- Elaborar historias de usuario y criterios de aceptación.
- Identificar dependencias entre módulos.
- Revisar que las funcionalidades cumplan los requerimientos definidos.
- Apoyar al equipo en el análisis de errores y cambios solicitados.

2 Product Owner (PO)

El Product Owner será responsable de maximizar el valor del producto y mantener el Product Backlog organizado y priorizado.

Sus responsabilidades serán:

- Definir y comunicar el objetivo del producto.
- Priorizar las historias según su valor para el gimnasio.
- Aclarar dudas sobre el comportamiento esperado del sistema.
- Aceptar o rechazar las funcionalidades según los criterios de aceptación.
- Mantener actualizado el backlog.
- Coordinar la validación de las entregas con los interesados.

3. Equipo de desarrollo

Será responsable de diseñar, programar, integrar y probar los componentes de Modulo membresia.

Sus responsabilidades serán:

- Implementar las historias priorizadas.
- Desarrollar las interfaces y la lógica de negocio.
- Integrar SQL Server y los servicios externos necesarios.
- Ejecutar pruebas y corregir defectos.
- Documentar los cambios realizados.
- Mantener actualizado el estado de las tareas.

3. Organización del trabajo ágil

Se propone trabajar en iteraciones de dos semanas. La duración podrá ajustarse a las fechas de entrega académicas y a la disponibilidad del equipo.

Actividad	Propósito	Resultado
Planificación del Sprint	**Seleccionar las historias que se trabajarán.**	**Objetivo e historias comprometidas.**
Desarrollo diario	**Revisar avances, bloqueos y siguientes pasos.**	**Trabajo coordinado.**
Revisión del Sprint	**Demostrar las funcionalidades desarrolladas.**	**Retroalimentación y validación.**
Retrospectiva	**Identificar mejoras para el equipo.**	**Acciones de mejora.**
Refinamiento del backlog	**Aclarar y dividir historias futuras.**	**Historias listas para planificar.**

El tablero Kanban permitirá visualizar el estado de cada historia con las columnas:

Pendiente → En progreso → En revisión → Terminado

Una tarea solo pasará a «Terminado» cuando cumpla con la Definition of Done establecida en este documento.

4. Product Backlog

El siguiente backlog es una propuesta inicial. Las prioridades podrán modificarse según las necesidades del gimnasio y los avances del equipo.

ID	Historia o funcionalidad	Prioridad	Estimación inicial
HU-001	**Inicio de sesión y control de acceso**	**Alta**	**5 puntos**
HU-002	**Registrar y consultar clientes**	**Alta**	**5 puntos**
HU-003	**Administrar planes del gimnasio**	**Alta**	**5 puntos**
HU-004	**Registrar y controlar membresías**	**Alta**	**8 puntos**
HU-005	**Registrar y consultar pagos**	**Alta**	**8 puntos**
HU-006	**Integrar pasarela de pago**	**Media**	**8 puntos**
HU-007	**Consultar indicadores del Dashboard**	**Media**	**5 puntos**
HU-008	**Generar reportes administrativos**	**Media**	**5 puntos**

5. Historias de usuario con las 3C

- Card (Tarjeta): descripción breve de la necesidad del usuario.
- Conversation (Conversación): reglas, detalles y decisiones que aclaran cómo debe funcionar la historia.

- Confirmation (Confirmación): criterios verificables para aceptar la funcionalidad.

HU-001. Inicio de sesión y control de acceso

Como administrador del gimnasio, quiero iniciar sesión con mis credenciales para acceder de forma segura a las funciones.

Conversation

- El sistema solicitará usuario o correo electrónico y contraseña.
- Las credenciales deberán validarse antes de permitir el acceso.
- Las contraseñas deberán almacenarse mediante un mecanismo seguro de hash.
- El acceso a las funcionalidades deberá respetar los permisos del usuario.
- La sesión deberá cerrarse cuando el usuario seleccione la opción de salir.

Confirmation

1. Dado un usuario activo con credenciales válidas, cuando inicia sesión, entonces el sistema le permite acceder al panel principal.
2. Dadas unas credenciales incorrectas, cuando intenta iniciar sesión, entonces el sistema deniega el acceso y muestra un mensaje comprensible.
3. Dado un campo obligatorio vacío, cuando se intenta iniciar sesión, entonces el sistema solicita completar la información.
4. Cuando el usuario cierra sesión, entonces el sistema vuelve a la pantalla de autenticación.

HU-002. Registro y consulta de clientes

Como recepcionista, quiero registrar y consultar clientes para mantener actualizada la información de las personas inscritas en el gimnasio.

Conversation

- El formulario permitirá introducir los datos personales definidos para el cliente.
- Los campos obligatorios deberán validarse.
- El sistema permitirá buscar y filtrar clientes.
- Se deberá evitar la duplicación de clientes según las reglas de identificación definidas.
- La información se almacenará en SQL Server.

Confirmation

1. Dada información válida, cuando el recepcionista guarda un cliente, entonces el registro queda almacenado y aparece en el listado.
2. Dado un formulario incompleto, cuando se intenta guardar, entonces el sistema informa los campos que deben corregirse.
3. Dado un cliente existente, cuando se realiza una búsqueda, entonces el sistema presenta sus datos.

4. Cuando no existen coincidencias, entonces el sistema informa que no se encontraron resultados.

HU-003. Administración de planes

Como administrador, quiero crear y modificar los planes del gimnasio para mantener actualizados los precios y las duraciones ofrecidas a los clientes.

Conversation

- Cada plan tendrá nombre, descripción, precio, duración y estado.
- El precio deberá ser mayor que cero.
- La duración deberá ser válida.
- Los planes desactivados no podrán utilizarse para nuevas contrataciones.
- Los planes asociados a registros existentes deberán conservar la integridad de la información histórica.

Confirmation

1. Cuando se registra un plan con datos válidos, entonces aparece en el listado de planes.
2. Cuando se introduce un precio no válido o una duración incorrecta, entonces el sistema rechaza el registro.
3. Cuando se modifica un plan, entonces los nuevos datos quedan guardados.
4. Cuando un plan se desactiva, entonces deja de estar disponible para nuevas membresías.

HU-004. Gestión de membresías

Como recepcionista, quiero registrar y consultar las membresías de los clientes para controlar sus períodos de vigencia y conocer cuáles están activas, próximas a vencer, vencidas o canceladas.

Conversation

- Cada membresía deberá estar asociada a un cliente y un plan existentes.
- El sistema registrará las fechas de inicio y finalización.
- La fecha de finalización deberá ser posterior a la fecha de inicio.
- La duración del plan podrá utilizarse para calcular la fecha final.
- El sistema deberá distinguir el estado de la membresía del estado de su pago.
- Las membresías deberán poder consultarse por cliente y estado.

Confirmation

1. Dado un cliente y un plan válidos, cuando se registra una membresía con fechas correctas, entonces el sistema guarda la información.
2. Dado un cliente o plan inexistente, cuando se intenta registrar una membresía, entonces el sistema rechaza la operación.
3. Cuando se consulta el listado por estado, entonces el sistema muestra únicamente las membresías que cumplen el filtro.
4. Cuando la fecha de vigencia ha finalizado, entonces la membresía se identifica como vencida según las reglas del sistema.
5. Una membresía no se considerará pagada únicamente por haber sido registrada.

HU-005. Registro y consulta de pagos

Como encargado de caja, quiero registrar y consultar los pagos de las membresías para mantener un historial confiable de las transacciones del gimnasio.

Conversation

- Cada pago deberá asociarse a la membresía correspondiente.
- El monto deberá ser válido y positivo.
- Se registrarán la fecha, el monto, la moneda y el estado del pago.
- Los pagos deberán distinguirse entre pendientes, aprobados y rechazados.
- Los reportes de ingresos solo deberán considerar los pagos confirmados.

Confirmation

1. Dado un pago con datos válidos, cuando se registra correctamente, entonces queda asociado a la membresía correspondiente.
2. Cuando se intenta registrar un monto inválido, entonces el sistema rechaza la operación.
3. Cuando se consulta el historial de una membresía, entonces se muestran sus pagos registrados.
4. Cuando un pago es rechazado, entonces no se contabiliza como ingreso confirmado.

HU-006. Integración con pasarela de pago

Como cliente del gimnasio, quiero realizar el pago electrónico de mi membresía para completar mi transacción mediante una pasarela de pago segura.

Conversation

- El sistema enviará el monto, la moneda y el token de pago requerido por el proveedor.
- La comunicación deberá realizarse mediante una conexión segura.
- El resultado de la transacción deberá registrarse.
- Los errores de comunicación deberán diferenciarse de los rechazos confirmados.
- Se deberán evitar los cobros duplicados cuando se reintente una operación.
- La integración dependerá de la disponibilidad y documentación del proveedor seleccionado.

Confirmation

1. Dada una solicitud válida, cuando la pasarela confirma el pago, entonces el sistema registra la transacción como aprobada.
2. Cuando la pasarela rechaza el pago, entonces el sistema registra el rechazo confirmado.
3. Cuando no se recibe una respuesta concluyente, entonces el sistema deja la operación pendiente de verificación y evita repetir el cobro sin comprobar su estado.
4. Cuando la solicitud es inválida, entonces el sistema informa el error correspondiente.
5. Los datos completos de la tarjeta no se almacenan en el modulo de membresias

HU-007. Dashboard de indicadores

Como administrador, quiero visualizar un resumen de clientes, ingresos, membresías por vencer y nuevas inscripciones para conocer el estado general del gimnasio.

Conversation

- Los indicadores se calcularán a partir de los registros almacenados.
- Los ingresos corresponderán a pagos confirmados.
- Las membresías por vencer se determinarán según un período definido.
- El panel deberá permitir acceder a los módulos relacionados.
- Los errores de consulta deberán comunicarse sin bloquear innecesariamente toda la interfaz.

Confirmation

1. Cuando el administrador abre el panel, entonces el sistema presenta los indicadores disponibles.
2. Cuando existen pagos confirmados, entonces el total de ingresos refleja esos pagos en el período seleccionado.
3. Cuando existen membresías próximas a vencer, entonces el panel presenta la cantidad correspondiente.
4. Cuando no hay registros, entonces el sistema muestra valores en cero o mensajes informativos.

HU-008. Reportes administrativos

Como administrador, quiero generar reportes de clientes, membresías y pagos por período para apoyar la toma de decisiones del gimnasio.

Conversation

- Los reportes permitirán seleccionar un rango de fechas cuando corresponda.
- Se validará que la fecha inicial no sea posterior a la fecha final.
- Los reportes mostrarán información coherente con los registros almacenados.
- La exportación a PDF o Excel estará disponible si se implementa en el sistema.
- El acceso a la información dependerá de los permisos del usuario.

Confirmation

1. Dado un rango de fechas válido, cuando se genera un reporte, entonces el sistema muestra los registros correspondientes.
2. Dado un rango de fechas inválido, cuando se solicita el reporte, entonces el sistema informa el problema.
3. Cuando no existen registros en el período, entonces el sistema presenta un resultado vacío o un mensaje informativo.
4. Cuando la exportación está habilitada, entonces el archivo generado contiene la información del reporte.

6. Validación de las historias mediante INVEST

El criterio INVEST permite evaluar la calidad de las historias de usuario:

Criterio	Significado	Aplicación en FitControl
I — Independent	**Independiente**	**Cada historia representa una funcionalidad delimitada. Las dependencias, como cliente y plan antes de membresía, se gestionan mediante el orden del backlog.**
N — Negotiable	**Negociable**	**Los detalles de implementación pueden discutirse con el equipo sin perder el objetivo del usuario.**
V — Valuable	**Valiosa**	**Cada historia aporta valor a la administración del gimnasio.**
E — Estimable	**Estimable**	**El equipo puede evaluar el esfuerzo a partir de las reglas y el alcance definidos.**
S — Small	**Pequeña**	**Cada historia debe poder completarse dentro de una iteración. Si resulta demasiado grande, se dividirá en historias menores.**
T — Testable	**Comprobable**	**Cada historia contiene criterios de aceptación verificables mediante pruebas.**

7. Definition of Ready (DoR)

La Definition of Ready establece las condiciones que una historia de usuario debe cumplir antes de ser seleccionada para desarrollo.

Una historia estará lista para iniciar cuando:

1. Tenga un identificador único.
2. Incluya una descripción en formato de historia de usuario.
3. Tenga su objetivo y valor para el usuario claramente definidos.
4. Incluya criterios de aceptación verificables.
5. Haya sido revisada y aclarada por el Analista o Product Owner.
6. Tenga una estimación inicial acordada por el equipo.
7. Sus dependencias técnicas y funcionales estén identificadas.
8. Disponga de los recursos necesarios para comenzar, como acceso a la base de datos, diseños o documentación de integración.
9. No tenga dudas críticas que impidan su implementación.
10. Su alcance sea suficientemente pequeño para trabajarse dentro de una iteración.

Si una historia no cumple estas condiciones, permanecerá en el backlog para refinamiento.

8. Definition of Done (DoD)

La Definition of Done establece las condiciones que una historia debe cumplir para considerarse terminada.

Una historia estará terminada cuando:

1. La funcionalidad se haya implementado de acuerdo con los criterios de aceptación.
2. El código se haya integrado con la versión correspondiente del proyecto.
3. Las validaciones de entrada y las reglas de negocio estén implementadas.
4. Se hayan ejecutado las pruebas pertinentes y se hayan corregido los defectos bloqueantes.
5. La funcionalidad se haya probado con la base de datos o los servicios correspondientes.
6. Los errores previsibles se gestionen mediante mensajes controlados y registros técnicos cuando corresponda.
7. No se expongan contraseñas, credenciales, tokens ni información sensible en mensajes o registros.
8. La interfaz sea coherente con el diseño del sistema y permita completar la operación.
9. La documentación técnica relacionada se haya actualizado.
10. Los criterios de aceptación hayan sido verificados y la funcionalidad esté disponible para revisión del Product Owner.

Una historia no se considerará terminada solo porque el código compile o porque la pantalla esté diseñada.

9. Tablero Kanban y seguimiento

El equipo utilizará un tablero Kanban en Jira o GitHub Projects para registrar las historias y visualizar su avance.

Columnas propuestas:

Columna	Descripción
Backlog	**Historias pendientes de priorización o refinamiento.**
Ready	**Historias que cumplen la Definition of Ready.**
In Progress	**Historias en desarrollo.**
In Review	**Historias en revisión de código, integración o pruebas.**
Done	**Historias que cumplen la Definition of Done.**

Cada tarjeta del tablero deberá contener, como mínimo, el identificador, el título, la prioridad, la persona responsable, los criterios de aceptación y los enlaces a la documentación relacionada.

Enlace al tablero del proyecto: https://github.com/users/J0seph2000/projects/1

10. Organización inicial del backlog en iteraciones

La siguiente distribución es una propuesta de planificación, no un registro de trabajo ya ejecutado.

Iteración propuesta	Objetivo	Historias candidatas
Sprint 1	**Establecer acceso y administración básica.**	**HU-001, HU-002**
Sprint 2	**Configurar la oferta comercial del gimnasio.**	**HU-003, HU-004**
Sprint 3	**Gestionar transacciones y validar el flujo de pago.**	**HU-005, HU-006**
Sprint 4	**Completar indicadores y consultas administrativas.**	**HU-007, HU-008**

Módulo de Membresías de Gimnasio: Evaluación Arquitectónica

Componente 2: ISO/IEC 25010, ADRs (ISO/IEC/IEEE 42010), Modelo C4 (niveles 1-4) y Persistencia Políglota

1. Descripción del sistema, stakeholders y concerns (ISO 42010)

El Módulo de Membresías se encarga de todo lo que ocurre entre un socio y el gimnasio desde el punto de vista económico y de acceso: desde que la persona elige un plan hasta que cruza el torniquete o decide darse de baja. Incluye el catálogo de planes, la inscripción, el cobro recurrente, la renovación, el congelamiento, la cancelación, la validación de acceso a las instalaciones y los reportes.

Alcance funcional:

- **Alta y consulta de socios.**
- **Catálogo de planes (mensual, trimestral, anual, pase diario, familiar, corporativo) con promociones.**
- **Inscripción a un plan y cobro inicial.**
- **Renovación automática y reintentos de cobro.**
- **Congelamiento y cancelación de membresías.**
- **Validación de acceso en torniquetes (QR o lector biométrico).**
- **Notificaciones (vencimiento, cobro fallido, bienvenida).**
- **Reportes de ingresos, retención y ocupación.**

1.1 Stakeholders y concerns

ID	Stakeholder	Concerns principales	Viewpoint que los atiende
S1	Socio	Acceso rápido, cobros correctos, privacidad de sus datos, autogestión desde la app	Contexto, Contenedores, Seguridad
S2	Recepcionista	Alta rápida de socios, cobro en mostrador, consulta de estado	Componentes, Funcional
S3	Gerente de sede	Ocupación, ingresos, morosidad, retención	Datos, Reportes
S4	Finanzas / Contabilidad	Conciliación de pagos, facturación electrónica, trazabilidad	Datos, Integración
S5	Equipo de TI / DevOps	Despliegue simple, observabilidad, costo operativo	Despliegue

S6	**Desarrolladores**	**Mantenibilidad, testabilidad, claridad de módulos**	**Componentes, Código**
S7	**Dueño / Dirección**	**Costo, escalabilidad a nuevas sedes, tiempo de salida al mercado**	**Contexto, ADRs**
S8	**Proveedores externos (pasarela, torniquetes, mensajería)**	**Contratos de integración estables**	**Contenedores, Integración**

1.2 Viewpoints y vistas utilizadas

Viewpoint	Vista	Sección
Contexto	**C4 nivel 1**	**4.1**
Estructura de contenedores	**C4 nivel 2**	**4.2**
Estructura lógica	**C4 nivel 3**	**4.3**
Información / código	**C4 nivel 4 (clases y ER)**	**4.4**
Datos	**Persistencia políglota**	**5**
Decisiones	**ADRs**	**3**

2. Evaluación de calidad (ISO/IEC 25010)

Para evaluar la calidad usamos el modelo de producto de ISO/IEC 25010 en su versión de 2011, que define 8 características. Si se necesitara la versión 2023, el cambio es pequeño: *usabilidad* pasa a llamarse *interaction capability*, *portabilidad* se convierte en *flexibility* y se incorpora *safety*, que en este sistema no es una preocupación crítica.

2.1 Priorización de atributos

Característica	Prioridad	Justificación
Seguridad	**Alta**	**Datos personales, datos de pago y control de acceso físico**
Fiabilidad	**Alta**	**Un fallo en el acceso o en los cobros afecta directamente a los socios y a los ingresos**
Eficiencia de desempeño	**Alta**	**El torniquete debe responder en menos de 1 s y hay picos de ingreso en horas pico**

Adecuación funcional	**Alta**	**Las reglas de cobro, congelamiento y vigencia deben ser exactas**
Mantenibilidad	**Media**	**Las reglas de planes y promociones cambian con frecuencia**
Usabilidad	**Media**	**Recepción necesita flujos cortos y el socio debe autogestionarse**
Compatibilidad	**Media**	**Integración con pasarela, torniquetes y facturación**
Portabilidad	**Baja**	**Se despliega en un único proveedor cloud, aunque se usan contenedores**

2.2 Escenarios de calidad

ID	Atributo (subcaracterística)	Estímulo	Entorno	Respuesta esperada	Métrica
QA-01	**Eficiencia (comportamiento temporal)**	**El socio escanea su QR en el torniquete**	**Hora pico, 300 ingresos por minuto**	**El sistema valida vigencia y abre**	**p95 menor a 500 ms, p99 menor a 1 s**
QA-02	**Fiabilidad (disponibilidad)**	**Se cae la base de datos principal**	**Operación normal**	**El acceso sigue funcionando con la caché de vigencia**	**Disponibilidad del acceso de 99.9 %, degradación hasta 15 min sin PostgreSQL**
QA-03	**Fiabilidad (tolerancia a fallos)**	**La pasarela de pagos no responde durante un cobro**	**Ciclo de renovación**	**Se reintenta con backoff y el cobro no se duplica**	**0 cobros duplicados, hasta 3 reintentos en 72 h**
QA-04	**Seguridad (confidencialidad)**	**Un usuario intenta leer datos de otro socio**	**Producción**	**La API rechaza la petición por autorización**	**100 % de intentos denegados, evento auditado**

QA-05	**Seguridad (integridad)**	**Alguien intenta alterar el monto de un pago**	**Producción**	**Se detecta por firma y validación en servidor**	**0 modificaciones aceptadas, alerta en menos de 1 min**
QA-06	**Adecuación funcional (corrección)**	**Un socio congela su membresía 15 días**	**Mitad de ciclo**	**La vigencia se extiende exactamente 15 días y no se cobra en ese lapso**	**100 % de casos de prueba de reglas aprobados**
QA-07	**Mantenibilidad (modificabilidad)**	**Marketing crea una nueva promoción con regla propia**	**Desarrollo**	**Se configura sin cambios de esquema relacional**	**Menos de 1 día de trabajo y 0 migraciones de PostgreSQL**
QA-08	**Mantenibilidad (capacidad de prueba)**	**Un cambio en una regla de renovación**	**CI/CD**	**Las pruebas unitarias del dominio corren aisladas de la infraestructura**	**Cobertura de dominio mayor a 80 %, suite en menos de 3 min**
QA-09	**Usabilidad (operabilidad)**	**La recepcionista inscribe un socio nuevo**	**Mostrador**	**Completa alta, plan y cobro en un flujo**	**Menos de 2 min y menos de 6 pantallas**
QA-10	**Compatibilidad (interoperabilidad)**	**Se cambia el proveedor de torniquetes**	**Evolución**	**Solo cambia el adaptador, no el dominio**	**Menos de 1 componente de dominio modificado**
QA-11	**Eficiencia (capacidad)**	**Se abren 5 nuevas sedes**	**Crecimiento**	**El sistema soporta 5 veces la carga actual escalando horizontalmente**	**Hasta 25 000 socios activos sin rediseño**

2.3 Tácticas arquitectónicas por escenario

Escenario	Táctica	Elemento en la arquitectura
QA-01, QA-02	Caché, degradación elegante, precálculo	Redis con estado de acceso por socio, Servicio de Acceso independiente
QA-03	Reintentos, idempotencia, circuit breaker	Worker de facturación, claves de idempotencia, Saga de cobro
QA-04, QA-05	Autenticar, autorizar, validar firmas, auditar	Keycloak (OIDC), RBAC, webhooks firmados, bitácora de auditoría
QA-06	Dominio rico y pruebas de reglas	Módulo de dominio de Membresía
QA-07	Esquema flexible	MongoDB para el catálogo de planes y promociones
QA-08, QA-10	Arquitectura hexagonal, puertos y adaptadores	Adaptadores de pasarela y torniquetes
QA-09	Reducir pasos, valores por defecto	Flujo de alta unificado en la SPA de recepción
QA-11	Escalamiento horizontal, API sin estado	Contenedores replicables, broker de mensajes

2.4 Trade-offs y riesgos entre atributos

Trade-off	Decisión tomada	Riesgo residual
Desempeño del acceso vs. consistencia inmediata	Se usa caché en Redis que puede estar desactualizada unos segundos	Un socio con pago recién vencido entra una vez más. Se acepta, con TTL corto y invalidación por evento
Simplicidad operativa vs. independencia de despliegue	Monolito modular con workers separados en lugar de microservicios completos	Mayor acoplamiento de despliegue en el núcleo
Flexibilidad del catálogo vs. integridad	Planes en MongoDB, pero cada membresía guarda una copia (snapshot) del plan contratado en PostgreSQL	Hay que mantener sincronizado el esquema del snapshot
Seguridad vs. rapidez en mostrador	Autenticación con roles y sesión corta, sin segundo factor para recepción en mostrador	Suplantación de recepcionista, mitigada con bitácora de auditoría

Número de motores de datos vs. costo operativo	**4 motores, cada uno con un propósito claro**	**Mayor carga de operación y respaldo**

3. Decisiones arquitectónicas (ADRs)

Todos los ADR siguen el mismo formato: contexto, decisión, alternativas, consecuencias, atributos de calidad impactados y stakeholders/concerns atendidos, de modo que cada decisión se pueda rastrear hasta ISO 42010.

ADR-001: Monolito modular con workers separados

- **Estado: Aceptado**
- **Contexto: Somos un equipo pequeño (3 a 5 desarrolladores) con un dominio bien acotado, en el que inscripción, pagos y vigencia están muy ligados entre sí. Esperamos crecer a varias sedes, pero no hasta necesitar miles de servicios.**
- **Decisión: Construir un backend monolítico modular con los módulos Socios, Planes, Membresías, Pagos, Acceso y Notificaciones, y con límites estrictos entre ellos. A su lado funcionan procesos worker independientes para facturación y notificaciones, y el Servicio de Acceso se despliega por separado porque ahí la latencia es crítica.**
- **Alternativas: Consideramos microservicios completos, que escalan mejor de forma independiente, pero el costo operativo y de consistencia no se justifica para este tamaño. También evaluamos un monolito clásico sin módulos: arranca más rápido, pero con el tiempo deteriora la mantenibilidad.**
- **Consecuencias: A favor: menos complejidad y transacciones locales en el núcleo, además de un camino abierto para extraer servicios si algún día hace falta. En contra: el núcleo se despliega como una sola unidad y se necesita disciplina para respetar los límites entre módulos.**
- **Calidad: Mantenibilidad (QA-08), Eficiencia (QA-11), costo operativo.**
- **Stakeholders/concerns: S5, S6, S7 (costo, simplicidad, tiempo de salida).**

ADR-002: Comunicación híbrida: REST síncrono y eventos asíncronos con RabbitMQ

- **Estado: Aceptado**
- **Contexto: Las consultas de los clientes necesitan respuesta inmediata, pero los cobros, las notificaciones y la actualización de la caché pueden hacerse de forma asíncrona sin que nadie lo note.**
- **Decisión: Usamos REST/JSON para los clientes y las APIs externas, y publicamos los eventos de dominio (MembresiaActivada, PagoConfirmado, PagoFallido, MembresiaCongelada, MembresiaVencida) en RabbitMQ.**
- **Alternativas: Kafka es más potente, pero queda sobredimensionado para nuestro volumen. Hacerlo todo de forma síncrona ataría nuestra disponibilidad a la del integrante más lento.**

- **Consecuencias: A favor: desacopla los componentes, permite reintentos y RabbitMQ es sencillo de operar. En contra: aparece la consistencia eventual y hay que tolerar mensajes duplicados con consumidores idempotentes.**
- **Calidad: Fiabilidad (QA-03), Eficiencia, Compatibilidad.**
- **Stakeholders/concerns: S4, S5, S8.**

ADR-003: PostgreSQL como fuente de verdad transaccional

- **Estado: Aceptado**
- **Contexto: Socios, membresías, pagos y congelamientos necesitan integridad referencial y transacciones ACID, y Finanzas requiere poder rastrear cada movimiento.**
- **Decisión: PostgreSQL guarda socios, membresías, pagos, congelamientos y la tabla outbox.**
- **Alternativas: MySQL era una opción equivalente, aunque con menos funcionalidades como JSONB y particionado declarativo. Usar MongoDB para todo se descartó porque es débil en integridad para manejar dinero.**
- **Consecuencias: A favor: integridad fuerte y consultas SQL para los reportes. En contra: al crecer habrá que escalar verticalmente y apoyarse en réplicas de lectura.**
- **Calidad: Adecuación funcional (QA-06), Seguridad/integridad (QA-05), Fiabilidad.**
- **Stakeholders/concerns: S3, S4.**

ADR-004: Redis para validación de acceso y sesiones

- **Estado: Aceptado**
- **Contexto: El torniquete debe responder en menos de 1 s, incluso en hora pico y aunque el núcleo esté parcialmente caído.**
- **Decisión: Redis guarda la clave `acceso:{sociold}` con el estado de vigencia y la sede permitida, con un TTL corto. Se mantiene al día mediante los eventos (`MembresiaActivada`, `MembresiaVencida`, `MembresiaCongelada`) y también lo usamos para limitar la tasa de peticiones a la API.**
- **Alternativas: Consultar PostgreSQL en cada ingreso añadiría latencia y una dependencia crítica. Una caché local en el torniquete sería difícil de invalidar.**
- **Consecuencias: A favor: latencia de milisegundos y tolerancia a una caída de PostgreSQL. En contra: hay una ventana de inconsistencia de unos segundos y Redis necesita réplica y persistencia AOF.**
- **Calidad: Eficiencia (QA-01), Fiabilidad (QA-02).**
- **Stakeholders/concerns: S1, S2, S5.**

ADR-005: MongoDB para el catálogo de planes y promociones

- **Estado: Aceptado**
- **Contexto: Los planes tienen atributos muy distintos entre sí (clases incluidas, sedes, invitados, descuentos por convenio) y marketing los modifica con frecuencia.**
- **Decisión: Guardamos planes, promociones y convenios como documentos en MongoDB. Cuando un socio se inscribe, copiamos el plan como snapshot dentro de su membresía en PostgreSQL.**
- **Alternativas: JSONB en PostgreSQL era viable y más simple; lo descartamos únicamente porque marketing necesita esquema libre y versionado de documentos. Las tablas EAV las descartamos por lo difíciles que son de mantener.**
- **Consecuencias: A favor: los cambios de catálogo no requieren migraciones (QA-07) y el snapshot evita que afecten a los contratos vigentes. En contra: hay un motor más que operar.**
- **Calidad: Mantenibilidad (QA-07), Adecuación funcional.**
- **Stakeholders/concerns: S3, S6, S7.**

ADR-006: TimescaleDB para registros de acceso y ocupación

- **Estado: Aceptado**
- **Contexto: Cada ingreso al gimnasio genera un evento, así que los volúmenes crecen rápido, y casi siempre se consultan por rangos de tiempo (ocupación por hora, asistencia por socio).**
- **Decisión: Usamos TimescaleDB, una extensión de PostgreSQL, con hypertables, compresión y agregados continuos para calcular la ocupación por hora y por sede.**
- **Alternativas: InfluxDB nos obligaría a aprender otro lenguaje de consulta y a sumar otro ecosistema. Una tabla común en PostgreSQL se degradaría con el volumen.**
- **Consecuencias: A favor: consultas temporales eficientes, retención configurable y seguimos usando SQL. En contra: requiere una instancia separada del núcleo, aunque eso también ayuda a aislar la carga.**
- **Calidad: Eficiencia (QA-11), Mantenibilidad.**
- **Stakeholders/concerns: S3, S5.**

ADR-007: Patrón Outbox y Saga orquestada para cobros

- **Estado: Aceptado**
- **Contexto: Activar una membresía implica cobrar en la pasarela externa, escribir en PostgreSQL, actualizar Redis y notificar al socio, y no existe una transacción distribuida que cubra todos esos sistemas.**
- **Decisión: Cada evento se escribe en una tabla `outbox` dentro de la misma transacción que el cambio de estado, y un publicador se encarga de enviarlo a RabbitMQ. El cobro sigue una Saga orquestada en tres pasos: reservar, cobrar y activar. Si el cobro falla, se compensa cancelando la reserva. Además, enviamos claves de idempotencia a la pasarela.**

- **Alternativas: Hacer dos escrituras directas arriesga dejar los datos inconsistentes, y el commit en dos fases no está soportado por la pasarela.**
- **Consecuencias: A favor: no hay cobros duplicados ni eventos perdidos (QA-03). En contra: hay más código y aparecen estados intermedios como `PENDIENTE_PAGO`.**
- **Calidad: Fiabilidad, Adecuación funcional.**
- **Stakeholders/concerns: S1, S4.**

ADR-008: Autenticación con Keycloak (OIDC/JWT) y autorización RBAC

- **Estado: Aceptado**
- **Contexto: Hay tres tipos de usuarios (socio, staff y administrador) con permisos distintos, y el sistema maneja datos personales y de pago.**
- **Decisión: Keycloak será el proveedor de identidad (OIDC), con tokens JWT de vida corta y los roles `SOCIO`, `RECEPCION`, `GERENTE` y `ADMIN`. Además, la API verifica que cada recurso pertenezca a quien lo solicita. Nunca guardamos datos de tarjeta: usamos la tokenización de la pasarela.**
- **Alternativas: Construir nuestra propia autenticación implica riesgo y costo de mantenimiento, y usar solo API keys no alcanza para los socios finales.**
- **Consecuencias: A favor: es un estándar, tiene MFA disponible y reduce el alcance de PCI-DSS. En contra: es un componente más que operar.**
- **Calidad: Seguridad (QA-04, QA-05).**
- **Stakeholders/concerns: S1, S4, S5.**

ADR-009: Despliegue en contenedores con orquestación gestionada

- **Estado: Aceptado**
- **Contexto: El equipo de operación es pequeño y necesitamos escalar en horas pico y poder reproducir los ambientes sin sorpresas.**
- **Decisión: Empaquetar todo en imágenes Docker y desplegarlas en un servicio de contenedores gestionado (Kubernetes gestionado o equivalente). Las bases de datos serán servicios gestionados siempre que se pueda, y el CI/CD funcionará con pipelines automatizados.**
- **Alternativas: Los servidores virtuales manuales son poco reproducibles, y serverless tendría latencia por arranques en frío justo en el acceso, donde menos nos la podemos permitir.**
- **Consecuencias: A favor: escalamiento horizontal y despliegues repetibles. En contra: hay una curva de aprendizaje.**
- **Calidad: Portabilidad, Eficiencia (QA-11), Mantenibilidad.**
- **Stakeholders/concerns: S5, S7.**

4. Modelo C4 (niveles 1 a 4)

Los diagramas de esta sección se elaboraron con Mermaid siguiendo la notación C4.

4.1 Nivel 1: Diagrama de contexto

4.2 Nivel 2: Diagrama de contenedores

4.3 Nivel 3: Diagrama de componentes (API de Membresías)

4.4 Nivel 4: Código

Aquí nos centramos en las dos partes más críticas: el modelo de dominio (clases) y el esquema relacional (entidad-relación).

Modelo de dominio (clases):

Esquema relacional (PostgreSQL):

5. Estrategia de persistencia políglota

5.1 Motor por tipo de dato

Motor	Datos que almacena	Por qué este motor	Propietario (módulo)	Respaldo / disponibilidad
PostgreSQL	Socios, membresías, pagos, congelamientos, outbox	ACID, integridad referencial, reportes SQL, JSONB para el snapshot	API de Membresías	Réplica de lectura, respaldos diarios y WAL continuo
MongoDB	Planes, promociones, convenios corporativos	Esquema flexible y cambios frecuentes sin migraciones	Módulo Planes	Replica set de 3 nodos

Redis	**Estado de acceso por socio, sesiones, contadores de rate limit**	**Latencia de milisegundos y TTL nativo**	**Servicio de Acceso**	**Réplica y persistencia AOF**
TimescaleDB	**Eventos de ingreso, agregados de ocupación**	**Series de tiempo, compresión y agregados continuos con SQL**	**Servicio de Acceso**	**Respaldos diarios, retención de 24 meses (datos detallados)**

Por ahora descartamos Elasticsearch: la búsqueda de socios por nombre o documento se resuelve bien con índices de PostgreSQL. Lo reconsideraríamos si apareciera la necesidad de una búsqueda difusa a gran escala.

5.2 Propiedad de datos

- **Cada base de datos pertenece a un único módulo o servicio. Los demás solo pueden acceder a ella mediante su API o por eventos, nunca directamente a sus tablas o colecciones.**
- **La API de reportes lee de réplicas de PostgreSQL y de los agregados de TimescaleDB, para no cargar las bases operativas donde se escribe.**

5.3 Consistencia entre bases

Flujo	Mecanismo	Garantía
Cambio de estado en PostgreSQL y publicación de evento	**Outbox transaccional**	**Al menos una entrega, sin eventos perdidos**
Evento a Redis (vigencia de acceso)	**Consumidor idempotente que escribe `acceso:{sociold}` con TTL**	**Consistencia eventual en segundos, con TTL como red de seguridad**
Alta de membresía con cobro	**Saga orquestada: `PENDIENTE_PAGO`, cobro, `ACTIVA`, o compensación**	**Estado final siempre coherente y sin cobros duplicados**

Snapshot de plan	**Se copia el plan de MongoDB a PostgreSQL al contratar**	**La membresía no cambia si el catálogo cambia**
Registro de ingreso	**El Servicio de Acceso escribe en TimescaleDB de forma asíncrona**	**Si falla el registro, el ingreso no se bloquea y se reintenta**
Reconciliación	**Job nocturno que compara membresías activas en PostgreSQL con claves en Redis**	**Corrige desvíos acumulados**

5.4 Flujo resumido de alta con cobro

1. **La recepcionista envía la inscripción. La API consulta el plan en MongoDB, crea la membresía en estado `PENDIENTE_PAGO` con el snapshot del plan y escribe el evento en el outbox, todo en la misma transacción de PostgreSQL.**
2. **El Worker de facturación le pide el cobro a la pasarela usando una clave de idempotencia.**
3. **La pasarela responde mediante un webhook firmado. Si el pago se confirma, la membresía pasa a `ACTIVA` y se emite `MembresiaActivada`.**
4. **El Servicio de Acceso consume ese evento y guarda la vigencia en Redis, mientras el Worker de notificaciones envía el mensaje de bienvenida.**
5. **Si el pago falla, se compensa: la membresía pasa a `CANCELADA` o se reintenta según la política definida, y se notifica al socio.**

6. Evaluación final

6.1 Matriz de trazabilidad: atributos, decisiones y vistas

Escenario	Atributo ISO 25010	ADRs	Vistas C4	Stakeholders
QA-01	**Eficiencia de desempeño**	**004, 009**	**Nivel 2 (Servicio de Acceso, Redis)**	**S1, S2**
QA-02	**Fiabilidad**	**004, 002**	**Nivel 2**	**S1, S5**
QA-03	**Fiabilidad**	**002, 007**	**Nivel 2, Nivel 3 (PasarelaAdapter)**	**S4**
QA-04	**Seguridad**	**008**	**Nivel 3 (AutorizacionFilter)**	**S1, S4**
QA-05	**Seguridad**	**003, 008, 007**	**Nivel 3 (Webhook), Nivel 4 (Pago)**	**S4**
QA-06	**Adecuación funcional**	**001, 003**	**Nivel 3 (CongelamientoService), Nivel 4**	**S1, S3**

QA-07	**Mantenibilidad**	**005**	**Nivel 2 (MongoDB), Nivel 3 (PlanRepository)**	**S3, S6**
QA-08	**Mantenibilidad**	**001**	**Nivel 3**	**S6**
QA-09	**Usabilidad**	**001, 002**	**Nivel 1, Nivel 2 (Back-office)**	**S2**
QA-10	**Compatibilidad**	**002, 001**	**Nivel 3 (adaptadores)**	**S5, S8**
QA-11	**Eficiencia / escalabilidad**	**001, 006, 009**	**Nivel 2 (TimescaleDB, contenedores)**	**S3, S5, S7**

6.2 Evaluación por característica ISO 25010

Característica	Cobertura	Evidencia
Adecuación funcional	**Cubierta**	**Dominio rico (Membresia, Congelamiento), Saga y pruebas de reglas**
Eficiencia de desempeño	**Cubierta**	**Caché en Redis, Servicio de Acceso aislado, TimescaleDB**
Compatibilidad	**Cubierta**	**Adaptadores y eventos de dominio**
Usabilidad	**Parcial**	**Se define el flujo objetivo, pero requiere pruebas con usuarios reales**
Fiabilidad	**Cubierta**	**Outbox, reintentos, idempotencia, degradación elegante**
Seguridad	**Cubierta**	**OIDC, RBAC, webhooks firmados, tokenización de pagos, auditoría**
Mantenibilidad	**Cubierta**	**Monolito modular, hexagonal, catálogo desacoplado**
Portabilidad	**Parcial**	**Contenedores, pero con servicios gestionados específicos de un proveedor**

6.3 Riesgos y puntos de sensibilidad (estilo ATAM)

ID	Riesgo	Impacto	Mitigación
R1	**Caída de Redis deja al Servicio de Acceso sin datos**	**Alto**	**Réplica, y modo de contingencia que consulta directamente PostgreSQL**
R2	**Eventos duplicados o fuera de orden en la caché**	**Medio**	**Consumidores idempotentes con versión del evento**

R3	**Complejidad operativa de 4 motores de datos**	**Medio**	**Servicios gestionados, monitoreo unificado, runbooks**
R4	**Los límites entre módulos del monolito se erosionan**	**Medio**	**Pruebas de arquitectura automáticas (por ejemplo, ArchUnit)**
R5	**Dependencia de la disponibilidad de la pasarela**	**Alto**	**Circuit breaker, reintentos y estados de cobro pendiente**
R6	**Tratamiento de datos personales y biométricos**	**Alto**	**Minimización, cifrado en reposo y tránsito, cumplimiento de la ley local de protección de datos**

6.4 Puntos de sensibilidad

- **TTL de la caché de acceso: un TTL corto mejora la exactitud de la información, pero aumenta la carga sobre PostgreSQL. Con un TTL largo pasa lo contrario.**
- **Número de reintentos de cobro: cada reintento adicional ayuda a recuperar ingresos, pero también prolonga el acceso de los socios morosos.**

4. DISEÑO DE APIS, IMPLEMENTACIÓN FUNCIONAL Y BDD (PASOS 3 Y 7)

4.1 Filosofía API-First y Contrato OpenAPI 3.0.3

El desarrollo del backend de FitControl se basa en la filosofía API-First, que consiste en definir y documentar el contrato de la interfaz de programación de aplicaciones (API) antes de utilizarla desde otros componentes del sistema. Este enfoque permite establecer las rutas disponibles, los métodos HTTP, los parámetros, los esquemas de datos, los mecanismos de autenticación y las respuestas esperadas.

Para documentar el contrato se utiliza OpenAPI 3.0.3, mediante un archivo YAML ubicado en docs/api/openapi.yaml. Este archivo describe las operaciones disponibles en el backend y sirve como referencia para los desarrolladores, las herramientas de pruebas y los consumidores de la API.

La documentación se visualiza mediante Swagger UI, disponible en http://localhost:3000/api-docs cuando el servidor está en ejecución. Esta herramienta permite consultar los endpoints, revisar sus parámetros y ejecutar solicitudes HTTP para verificar su comportamiento.

La implementación utiliza las siguientes tecnologías:

- Node.js y Express: ejecución del servidor y gestión de las solicitudes HTTP.
- MySQL y mysql2: almacenamiento y consulta de información persistente.
- JWT (JSON Web Token): autenticación de solicitudes protegidas.
- bcryptjs: protección de las contraseñas mediante funciones de hash.
- Swagger UI: visualización y prueba interactiva del contrato OpenAPI.
- Cucumber.js y Gherkin: definición y ejecución de pruebas de comportamiento.

El diseño del backend está organizado por capas, separando controladores, servicios, repositorios, DTOs y middleware. Esta separación facilita el mantenimiento, las pruebas y la modificación de las funcionalidades sin concentrar toda la lógica en un único archivo.

4.2 Estándar de Errores RFC 9110 y RFC 9457 (Problem Details)

FitControl utiliza los lineamientos de HTTP definidos en RFC 9110 para las respuestas y los códigos de estado, junto con el formato Problem Details establecido en RFC 9457 para representar los errores de manera estructurada.

El formato de respuesta utilizado para estos errores es application/problem+json. Su finalidad es proporcionar información uniforme que permita identificar la naturaleza del problema, el código HTTP correspondiente y los detalles necesarios para comprender la respuesta.

Un ejemplo de error de validación al registrar un cliente es el siguiente:

```
{
  "type": "https://fitcontrol.example/problems/validation-error",
  "title": "Solicitud inválida",
  "status": 400,
  "detail": "Los datos del cliente no son válidos.",
```

```
  "instance": "/api/clientes",
  "errors": [
    {
      "campo": "apellido",
      "mensaje": "apellido es obligatorio."
    }
  ]
}
```

En este ejemplo, el código HTTP 400 indica que la solicitud contiene datos inválidos. El campo title proporciona una descripción breve, detail explica el problema, instance identifica la ruta relacionada con el error y errors contiene los detalles específicos de validación.

Los campos type, title, status, detail e instance corresponden a los miembros estándar de Problem Details. El campo errors es una extensión específica de la aplicación para describir errores de validación. La dirección utilizada en type es ilustrativa y debe sustituirse por la dirección definitiva del proyecto si se publica el contrato.

Los principales códigos de estado contemplados en el diseño son:

Código HTTP	Significado	Aplicación en FitControl
400	Solicitud incorrecta	Datos de entrada que no cumplen las validaciones.
401	No autenticado	Token ausente, inválido, mal formado o vencido.
404	Recurso no encontrado	Cliente u otro recurso que no existe.
409	Conflicto	Operación incompatible con una restricción del sistema.
422	Entidad no procesable	Referencia inexistente, según las reglas de la aplicación.
500	Error interno	Fallo inesperado del servidor, sin exponer información sensible.

Para los endpoints protegidos se utiliza autenticación mediante JWT. Las respuestas 401 asociadas a autenticación Bearer incluyen el encabezado WWW-Authenticate: Bearer, conforme al esquema documentado.

La estandarización permite que los clientes de la API identifiquen los errores de forma consistente y evita que las respuestas expongan detalles internos innecesarios.

4.3 Catálogo de Endpoints Funcionales Desarrollados

El backend de FitControl proporciona endpoints REST para comprobar el estado del servicio, autenticar usuarios, administrar clientes, consultar planes y gestionar membresías.

La siguiente tabla presenta una selección de operaciones implementadas en el proyecto. Las rutas corresponden a la estructura documentada del backend y no deben confundirse con las rutas /api/v1/recursos del ejemplo genérico de la rúbrica.

Método HTTP	Endpoint API	Descripción	Base de datos / caché	Estado
GET	/api/health	Comprueba el estado de la API.	No requiere persistencia de negocio.	Implementado; verificar ejecución.
POST	/api/auth/register	Registra un usuario y protege su contraseña mediante hash.	MySQL.	Implementado; verificar ejecución.
POST	/api/auth/login	Valida credenciales y genera un token JWT.	MySQL.	Implementado; verificar ejecución.
GET	/api/clientes	Consulta clientes con filtros opcionales por estado y búsqueda.	MySQL.	Implementado; verificar ejecución.
POST	/api/clientes	Registra un nuevo cliente.	MySQL.	Implementado; verificar ejecución.
GET	/api/planes	Consulta el catálogo de planes.	MySQL y caché en memoria.	Implementado; verificar ejecución.
POST	/api/planes	Registra un plan e invalida la caché correspondiente.	MySQL y caché en memoria.	Implementado; verificar ejecución.
GET	/api/membresias	Consulta las membresías registradas.	MySQL.	Implementado; verificar ejecución.
POST	/api/membresias	Registra una membresía.	MySQL.	Implementado; verificar ejecución.

Las operaciones que requieren autenticación utilizan el token JWT en el encabezado Authorization, con el esquema Bearer. Los endpoints públicos de registro e inicio de sesión no requieren un token previo.

La persistencia se realiza mediante MySQL, mientras que la consulta de planes incorpora caché en memoria para reducir consultas repetidas. Por tanto, el proyecto no declara como

implementadas las tecnologías PostgreSQL, Redis, MongoDB o Neo4j de la tabla de referencia, ya que no corresponden al stack descrito para esta versión.

La funcionalidad de los endpoints se verifica mediante Swagger UI o Postman. Para demostrar la persistencia, se registra un cliente y posteriormente se consulta el listado para comprobar que el registro aparece en la respuesta.

Evidencia recomendada: incorporar capturas de Swagger UI mostrando la solicitud ejecutada, el código HTTP recibido y el cuerpo de respuesta. Para las operaciones de escritura, incluir evidencia de la consulta posterior que confirma que los datos quedaron almacenados.

4.4 Especificación BDD en Gherkin y Casos de Prueba de Aceptación (ISO/IEC/IEEE 29119-3:2021)

Las pruebas de comportamiento de FitControl se organizan mediante el enfoque BDD (Behavior-Driven Development), que permite expresar los requisitos del sistema como escenarios verificables. Los escenarios se redactan en Gherkin y se almacenan en la carpeta features/.

La estructura básica utiliza las palabras clave:

- Feature: funcionalidad que se desea verificar.
- Scenario: situación concreta que se va a probar.
- Scenario Outline: plantilla de escenario que se ejecuta con diferentes conjuntos de datos.
- Given: establece las condiciones iniciales.
- When: describe la acción que realiza el usuario o cliente.
- Then: define el resultado esperado.
- Examples: proporciona los datos utilizados por un Scenario Outline.

Como ejemplo ilustrativo de un escenario de autenticación, se presenta la siguiente especificación:

```
Feature: Autenticación de usuarios

  Scenario: Inicio de sesión con credenciales válidas
    Given que existe un usuario registrado en FitControl
    When se envían credenciales válidas al endpoint de login
    Then la API responde con el código HTTP 200
    And la respuesta contiene un token JWT
```

Este escenario define las condiciones previas, la operación y el resultado esperado. Para utilizarlo como evidencia de ejecución, debe corresponder a los pasos implementados en el proyecto y a los resultados observados durante las pruebas.

Los casos de prueba se documentan tomando como referencia la norma ISO/IEC/IEEE 29119-3:2021, relacionada con la documentación de pruebas de software. La información

de cada caso permite identificar qué se evalúa, cómo se ejecuta y cuál es el resultado esperado.

ID	Caso de prueba	Procedimiento	Resultado esperado
CP-01	Comprobar estado de la API	Ejecutar GET /api/health.	Respuesta HTTP satisfactoria.
CP-02	Inicio de sesión válido	Enviar credenciales correctas a POST /api/auth/login.	Respuesta satisfactoria y token JWT.
CP-03	Token inválido	Consultar un endpoint protegido con un token inválido.	Respuesta HTTP 401.
CP-04	Crear cliente	Enviar datos válidos a POST /api/clientes con autenticación.	Cliente creado y almacenado.
CP-05	Validación de cliente	Enviar datos incompletos a POST /api/clientes.	Error de validación con formato Problem Details.
CP-06	Consultar planes	Ejecutar GET /api/planes y revisar la información de caché cuando corresponda.	Respuesta satisfactoria y comportamiento de caché verificable.

Los casos anteriores constituyen ejemplos de documentación y deben contrastarse con las pruebas efectivamente implementadas y ejecutadas.

El proyecto contempla dos niveles principales de pruebas:

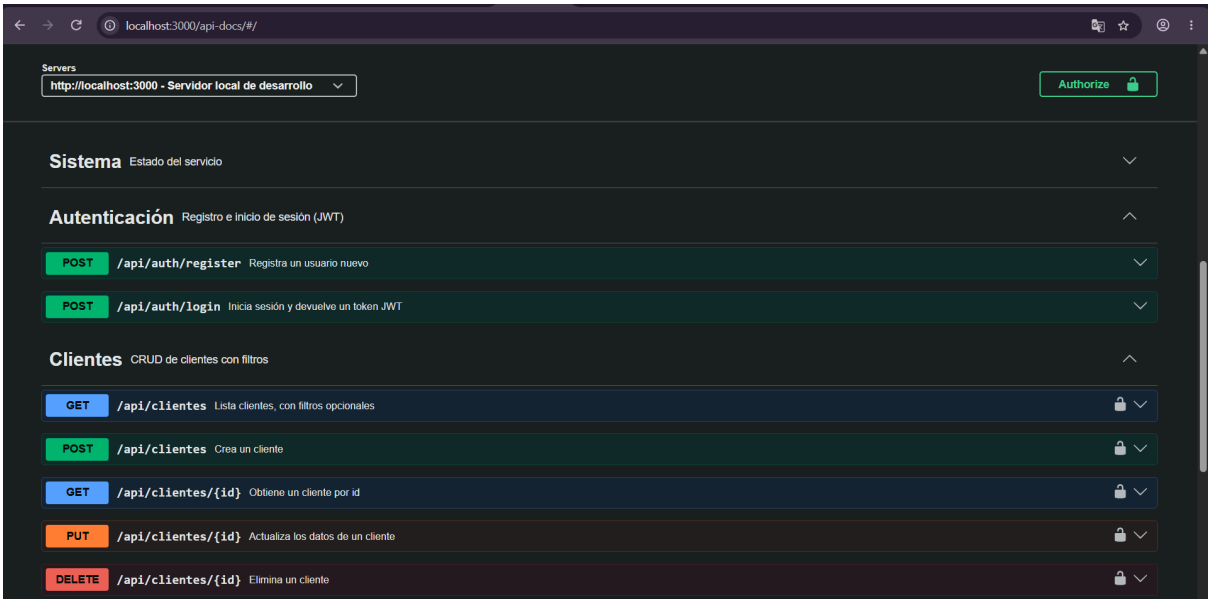
Pruebas unitarias: se ejecutan mediante npm test. Según la documentación del proyecto, incluyen 21 pruebas sobre servicios, caché y manejo de errores, sin requerir conexión a la base de datos.

Pruebas BDD: se ejecutan mediante npm run test:bdd. Según la documentación del proyecto, comprenden 25 casos y 44 escenarios con ejemplos contra la API real. Para ejecutarlas es necesario mantener el servidor activo y disponer de la base de datos configurada.

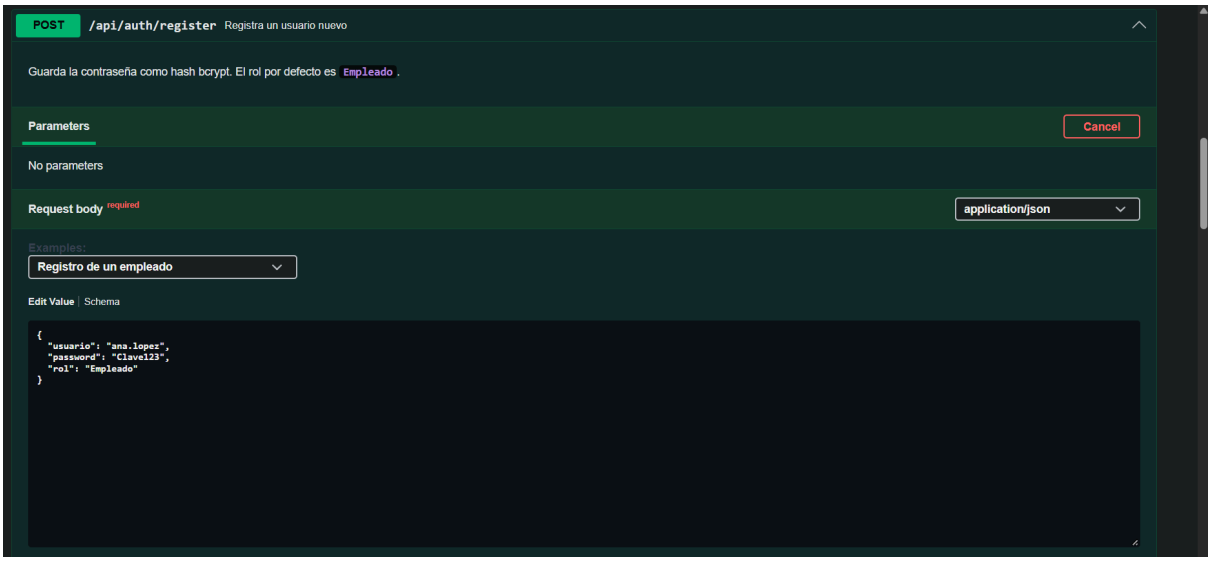
El reporte de las pruebas BDD se encuentra en tests/bdd/reporte-bdd.html, siempre que haya sido generado durante la ejecución.

Como evidencia documental se incluyen el contrato docs/api/openapi.yaml, los archivos de escenarios de features/, los pasos de ejecución de tests/bdd/steps.js y los documentos de pruebas ubicados en docs/qa/. Estos elementos permiten relacionar el contrato de la API con su implementación y con los procedimientos utilizados para verificar el comportamiento del sistema.

Captura de Contrato OPENAPI



Endpoints funcionales



Curl

```
curl -X 'POST' \
  'http://localhost:3000/api/auth/register' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "usuario": "ana.lopez",
  "password": "Clave123",
  "rol": "Empleado"
}'
```

Request URL

```
http://localhost:3000/api/auth/register
```

Server response

Code	Details
409	Error: Conflict

Response body

```
{
  "type": "https://fitcontrol.example/problems/conflict",
  "title": "Conflicto con el estado actual del recurso",
  "status": 409,
  "detail": "Ese nombre de usuario ya existe.",
  "instance": "/api/auth/register"
}
```

Download

Response headers

```
access-control-allow-origin: *
connection: keep-alive
content-length: 197
content-type: application/problem+json; charset=utf-8
date: Sat,24 Oct 2026 06:21:24 GMT
etag: W/"c5-X0RYQ+6SNXh8NTHO0/OjC+FO6hs"
keep-alive: timeout=5
x-powered-by: Express
```

Responses

Code	Description
201	Usuario creado

Media type: application/json

Controls Accept header.

Example Value | Schema

```
{
  "mensaje": "Usuario creado correctamente.",
  "id_usuario": 1
}
```

400	Solicitud inválida (datos faltantes, formato incorrecto o filtro desconocido)

Media type: application/problem+json

Example Value | Schema

```
{
  "type": "https://fitcontrol.example/problems/validation-error",
  "title": "Solicitud inválida",
  "status": 400,
  "detail": "Los datos del cliente no son válidos.",
  "instance": "/api/clientes",
  "errors": [
    {
      "campo": "apellido",
      "mensaje": "apellido es obligatorio."
    }
  ]
}
```

409	La operación choca con el estado actual de los datos

Media type: application/problem+json

409	La operación choca con el estado actual de los datos

Media type: application/problem+json

Example Value | Schema

```
{
  "type": "https://fitcontrol.example/problems/conflict",
  "title": "Conflicto con el estado actual del recurso",
  "status": 409,
  "detail": "Ese nombre de usuario ya existe.",
  "instance": "/api/auth/register"
}
```

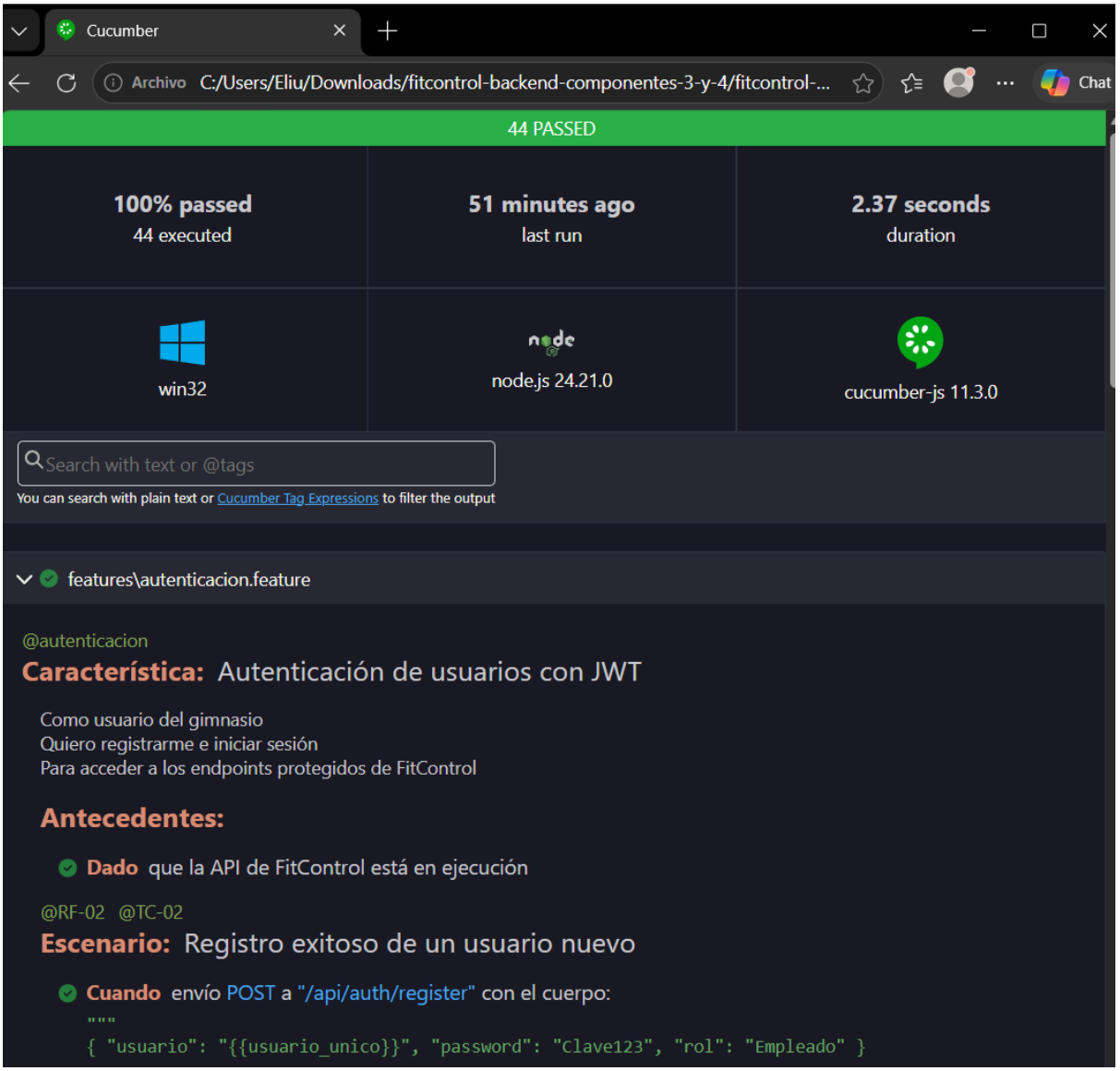

500	Error inesperado del servidor (sin detalles internos)

Media type: application/problem+json

Example Value | Schema

```
{
  "type": "https://fitcontrol.example/problems/internal-error",
  "title": "Error interno del servidor",
  "status": 500,
  "detail": "Ocurrió un error inesperado. Intenta de nuevo más tarde.",
  "instance": "/api/clientes"
}
```

Pruebas


Cucumber
Archivo C:/Users/Eliu/Downloads/fitcontrol-backend-componentes-3-y-4/fitcontrol-...
Chat
44 PASSED
100% passed
44 executed
51 minutes ago
last run
2.37 seconds
duration
win32
node.js 24.21.0
cucumber-js 11.3.0
Search with text or @tags
You can search with plain text or Cucumber Tag Expressions to filter the output
features\autenticacion.feature
@autenticacion
Característica: Autenticación de usuarios con JWT
Como usuario del gimnasio
Quiero registrarme e iniciar sesión
Para acceder a los endpoints protegidos de FitControl
Antecedentes:
Dado que la API de FitControl está en ejecución
@RF-02 @TC-02
Escenario: Registro exitoso de un usuario nuevo
Cuando envío POST a "/api/auth/register" con el cuerpo:
"""
{ "usuario": "{{usuario_unico}}", "password": "Clave123", "rol": "Empleado" }

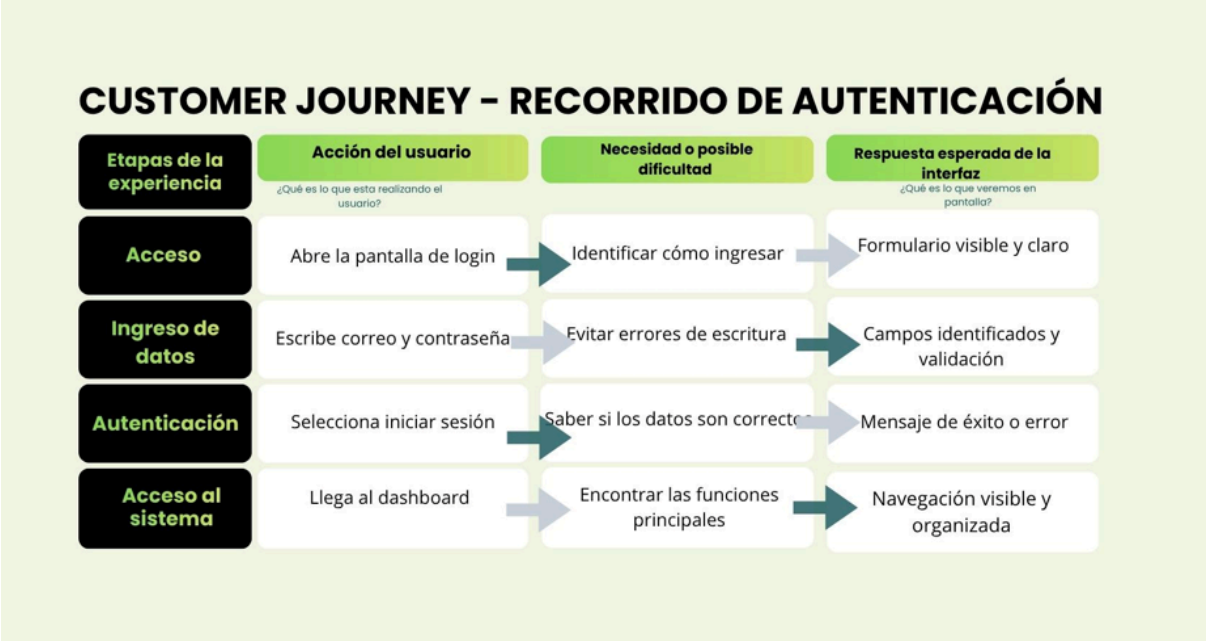
5. EXPERIENCIA DE USUARIO (UX/UI) Y MOBILE-FIRST

5.1 Diseño centrado en el usuario — ISO 9241-210

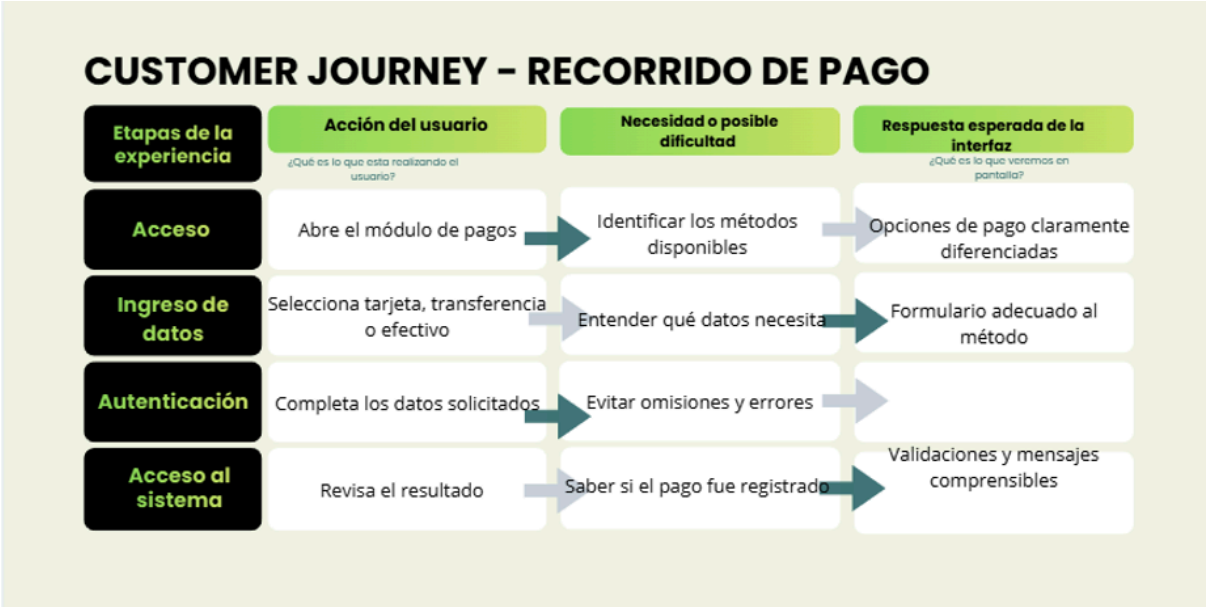
- Usuarios objetivo: administradores del gimnasio y personal encargado de clientes, membresías y pagos.
- Necesidades: encontrar rápidamente la información sobre clientes para poder brindar información cuando el cliente lo solicite, identificar las opciones de navegación, consultar planes y registrar pagos con claridad así mismo mantener un registro seguro de pagos y vencimiento de pagos.
- Contexto de uso: computadoras de escritorio, laptops y dispositivos móviles.
- Decisiones de diseño: menús claros, formularios organizados, botones identificables y estilos consistentes además de llamativos que adapten la estética de la marca.

Customer journey maps

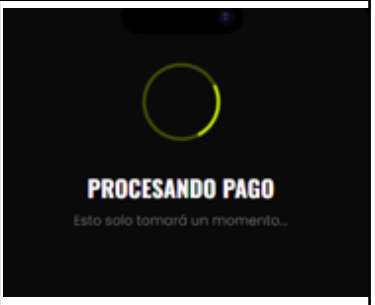
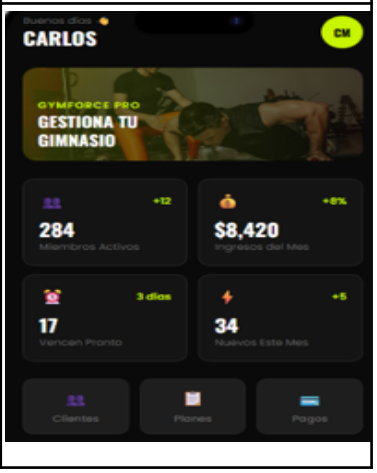
Este mapa describe los pasos por los que el usuario debe pasar para poder acceder al sistema, el cual se nombró recorrido de autenticación, este mapa ayuda para que se tenga previsto lo que se realizara para acceder al sistema.

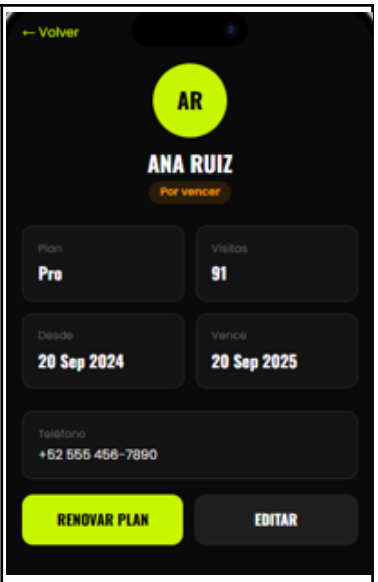
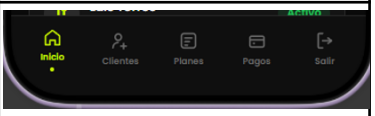
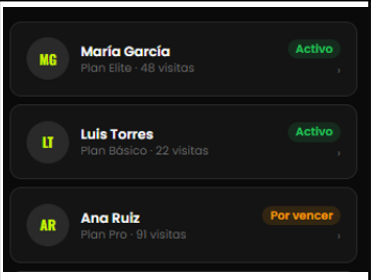


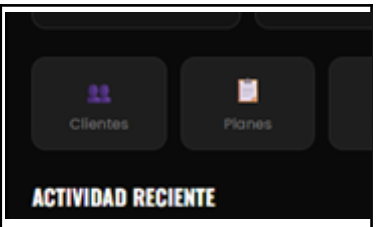
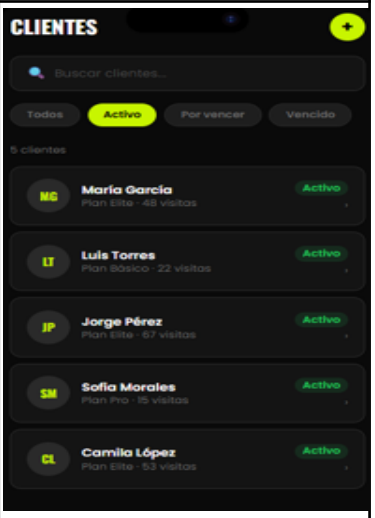
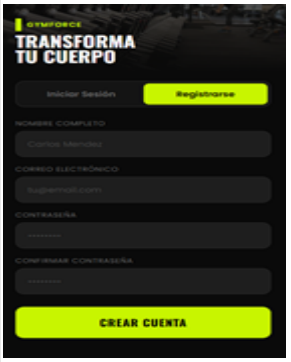
Este mapa describe el recorrido para realizar pago en el sistema, como tal el objetivo es que se pueda realizar el registro del pago del cliente y así mismo marcarlo como usuario activo



- **5.2 Evaluación Heurística (ISO 9241-11) y Accesibilidad WCAG 2.2 (POUR)**

Heurística	Aspecto que cumple	Descripción	Evidencia
1. Visibilidad del estado del sistema	Mensajes de carga, confirmación y error	✅Contiene mensajes que muestran lo que esta sucediendo para que sepa que el sistema esta en funcionamiento.	
2. Correspondencia con el mundo real	Términos comprensibles para la gestión de un gimnasio	✅Contiene términos que facilitan la gestión del gimnasio, como miembros activos, quienes vencen pronto el pago, esto para facilitar la gestión del gimnasio con los clientes.	

3. Control y libertad del usuario	Posibilidad de volver, cancelar o corregir una acción	✅Contiene botones que permiten volver a la pantalla anterior, así como poder editar si se registró mal a algún cliente o se desea editar su información.	
4. Consistencia y estándares	Botones, colores, iconos y navegación coherentes	✅Contiene botones con iconos coherentes de acuerdo con la acción que se realiza con ellos	
5. Prevención de errores	Validación de campos y confirmación de acciones importantes	✅El sistema valida campos necesarios u obligatorios y muestra mensajes de las validaciones que esta realizando, tal como la validación de que los campos de inicio de sesión estén llenos.	
6. Reconocimiento antes que recuerdo	Opciones y etiquetas visibles sin exigir memorizar información	✅Contiene etiquetas visibles que ayuden a identificar aspectos importantes y que se deben de considerar en el sistema, como las etiquetas del estado del pago de los clientes.	

7. Flexibilidad y eficiencia	Acceso sencillo a funciones frecuentes	✅ Contiene accesos fácilmente visibles para poder realizar funciones que se necesitan realizar constantemente.	
8. Diseño estético y minimalista	Pantallas ordenadas, sin elementos innecesarios	✅Contiene pantallas ordenadas para que la información este bien estructurada y sea sencillo leer para el usuario.	
9. Recuperación de errores	Mensajes que explican el problema y cómo corregirlo	✅Muestra mensajes cuando no se puede realizar una acción que indican que debe hacer el usuario para que pueda continuar.	
10. Ayuda y documentación	Instrucciones o indicaciones cuando sean necesarias	✅Contiene elementos visibles que facilitan que el usuario pueda ver lo que debe hacer.	

Resumen por principio (WCAG 2.2, nivel AA)

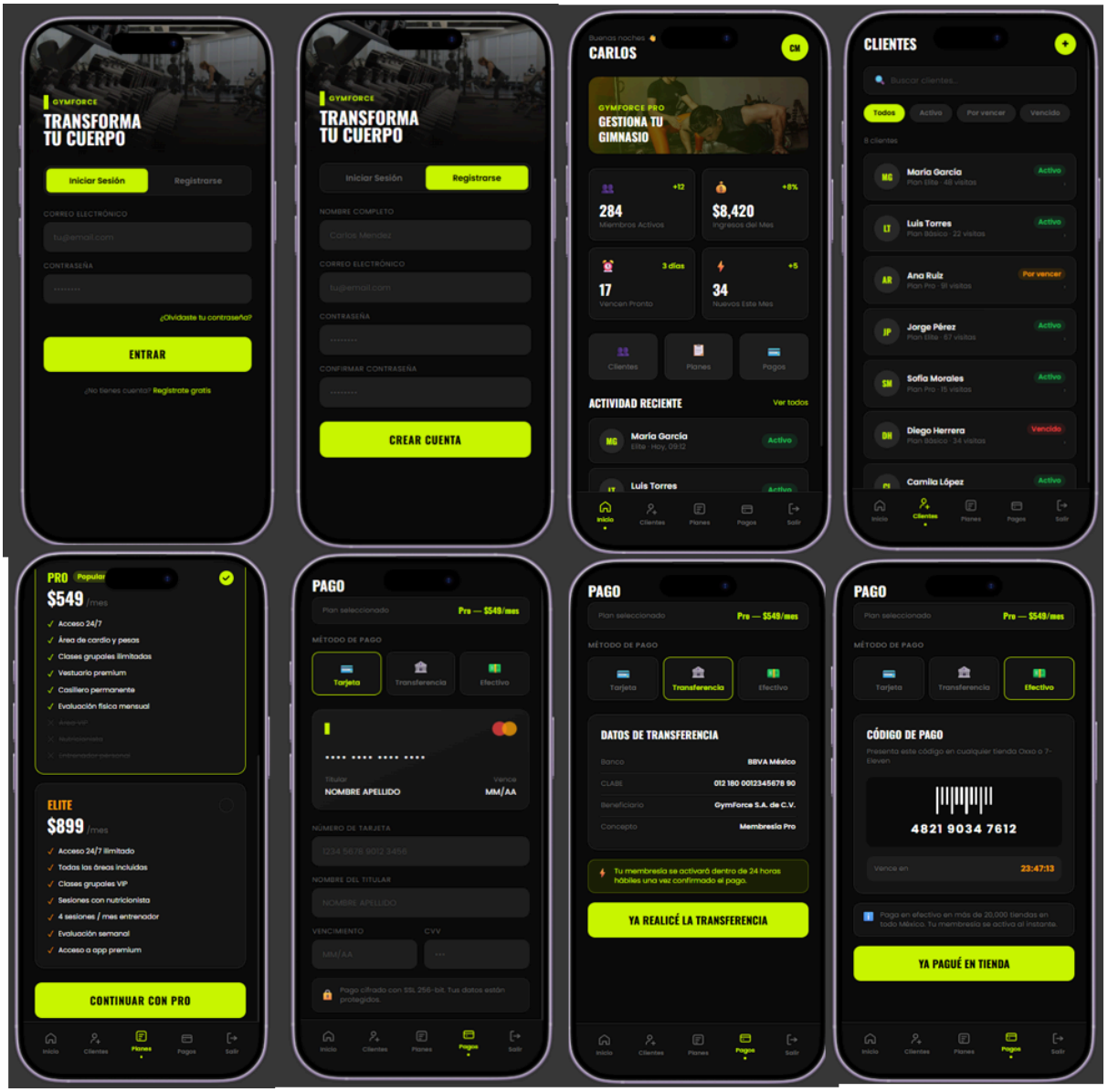
Principio	Estado general	Por qué
Perceptible	**Parcial**	Los botones principales (texto negro sobre verde neón) y los títulos blancos sobre fondo oscuro tienen contraste muy alto. Los estados no dependen solo del color: "Activo", "Por vencer" y "Vencido" llevan texto, el plan seleccionado lleva un ✓ y el menú inferior combina icono con etiqueta. Fallan los textos secundarios en gris sobre negro (etiquetas de campos, subtítulos de clientes, notas de seguridad).
Operable	**Parcial / No verificable**	Los botones de acción son grandes y fáciles de tocar, y la navegación inferior es consistente en todas las pantallas. No puedo confirmar el foco visible con teclado, el orden de tabulación ni el tamaño de objetivo de los chips de filtro y los iconos del menú, que se ven pequeños.
Comprensible	**Cumple en su mayoría**	Los textos están en español y las etiquetas son claras ("Correo electrónico", "Contraseña", "Nombre del titular"). Las pantallas siguen un patrón coherente. Hay aviso previo de la activación en 24 horas en transferencia, y el botón cambia de texto según el método de pago ("Ya realicé la transferencia", "Ya pagué en tienda"). También contiene mensajes para mostrar las validaciones que se realizan en los formularios.
Robusto	**No verificable**	Se estima que el sistema tenga la capacidad de poder aceptar grandes cantidades de usuarios y se pueda saber la información de los usuarios cuando estos la necesiten, también que se tenga acceso al sistema en las diferentes sucursales.

Estrategia Mobile-First y Arquitectura Visual con Atomic Design

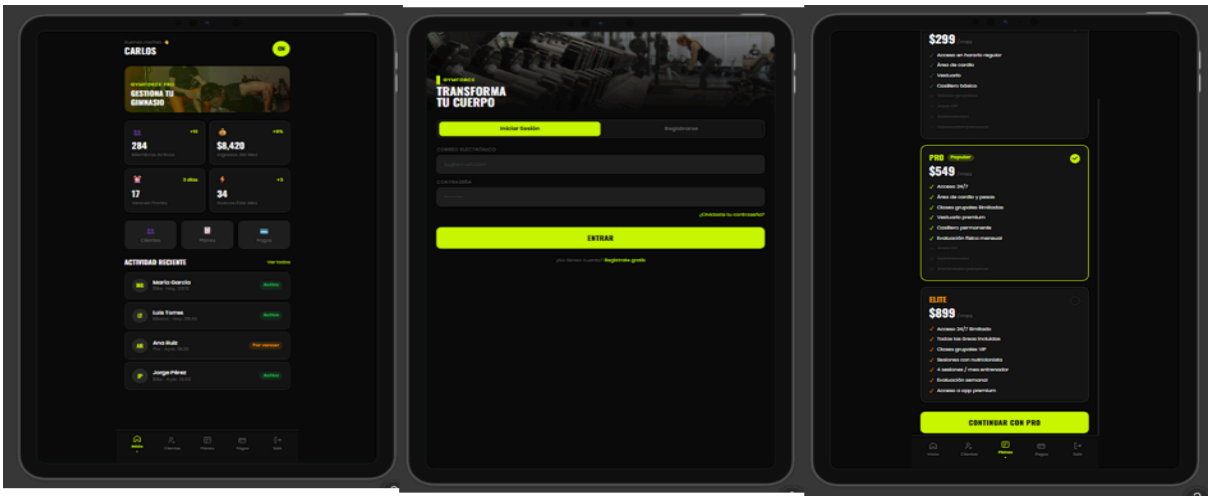
El diseño propuesto opto por la estrategia mobile-first, esto debido a las ventajas que se tenia en usabilidad, comodidad y más fácil contar con un dispositivo móvil a diferencia de contar con un dispositivo con pantallas más grandes, sin embargo en sistema tendría funcionamiento aunque fueran dispositivos con pantallas más amplias, aunque no aprovechara del todo el espacio de la pantalla.

Con la estrategia mobile-first, se logro obtener una interfaz de usuario amigable, agradable para el usuario y con facilidad de uso, contenía todas las pantallas adaptadas a las dimensiones de un dispositivo móvil ocupando el espacio de manera estratégica.

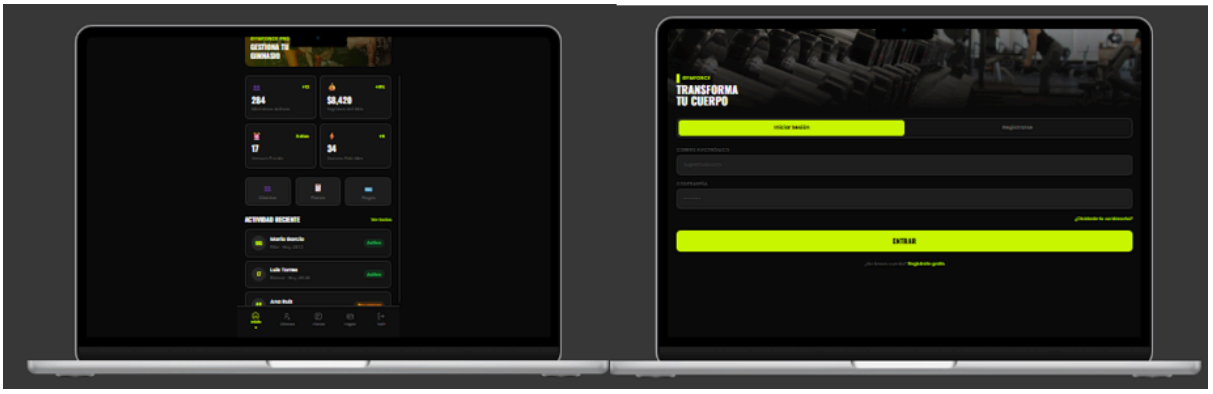
Al desarrollar como principal la interfaz de usuario con la estrategia mobile-first, no se tenia unas pantallas que se adaptaran directamente a las dimensiones de una pantalla más grande, manteniendo las dimensiones de un dispositivo móvil.



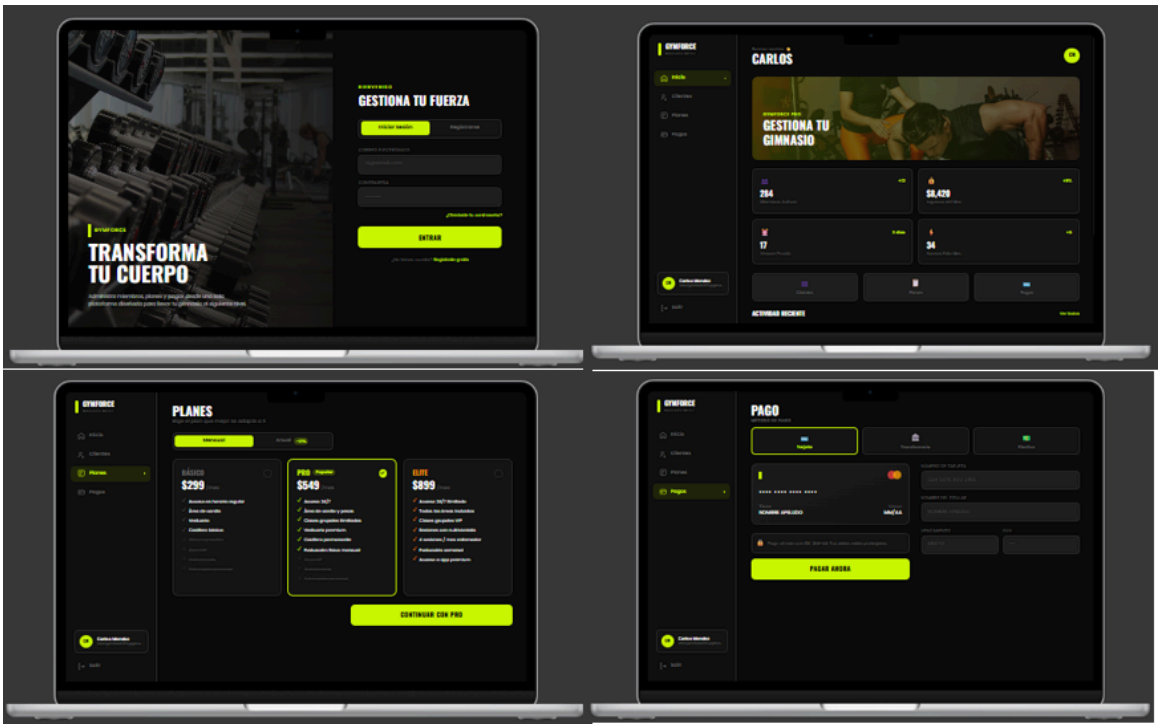
Tablet: Únicamente el login se adaptaba a los espacios, pero las demás pantallas no, aunque no lo hiciera de manera agradable, pero ya se tenia una interfaz funcional.



Escritorio



Por ultimo se implementaron los cambios en las pantallas como mejoras adaptadas al sistema.



. Atomic Design

Nivel	Ejemplos en FitControl
Átomos	**Colores, tipografías, iconos, etiquetas, botones e inputs**
Moléculas	**Campo de formulario con etiqueta y mensaje; buscador con botón; grupo de campos**
Organismos	**Sidebar, encabezado, formulario de login, tabla de clientes, tarjeta de plan y formulario de pago**
Plantillas	**Estructura general del dashboard y distribución de las pantallas de módulos**
Páginas	**Login, registro, dashboard, clientes, planes y pagos**

- Vínculo al Prototipo Interactivo: https://www.figma.com/make/dNaJtQyNjh48vgYfp66gto/Pantalla-m%C3%B3dulo-de-membres%C3%ADas?t=7g5ilio2yEfS7nxm-20&fullscreen=1

6. ESTRATEGIA DE CALIDAD Y PLAN SQA (PASO 10)

6.1 Plan de Aseguramiento de Calidad SQA (IEEE 730-2014) y Enfoque Shift-Left.

Alcance

Este plan comprende la evaluación de los módulos y artefactos asociados con FitControl, de acuerdo con su estado de implementación.

Autenticación: Validación de credenciales, mensajes de error y control de acceso

Dashboard: Claridad de la información y acceso a los módulos

Clientes: Validación de formularios y consistencia de las operaciones implementadas

Planes de membresía: Presentación y consistencia de la información de los planes

Pagos: Validación de datos, selección del método y comunicación del resultado

Interfaz web: Consistencia visual, navegación y adaptación a diferentes pantallas

Accesibilidad: Contraste, legibilidad, interacción y estructura de la interfaz

API y persistencia: Validación de solicitudes, respuestas, manejo de errores y almacenamiento, cuando estén implementados

Documentación: Coherencia entre requisitos, diseño, pruebas y resultados

El alcance de las pruebas deberá distinguir entre las funciones representadas en el prototipo de Figma y aquellas que se encuentren implementadas en el software. Las funcionalidades visuales no se considerarán verificadas a nivel de backend hasta que existan pruebas de ejecución que lo demuestren.

Normas y referencias de calidad

El proyecto utilizará las siguientes referencias para organizar las actividades de calidad:

Norma o referencia	Aplicación en FitControl
IEEE 730-2014	**Organización del Plan de Aseguramiento de Calidad del Software**
ISO 9241-210	**Diseño centrado en las necesidades de los usuarios**
ISO 9241-11	**Evaluación de la eficacia, eficiencia y satisfacción en el uso**
WCAG 2.2, nivel AA	**Criterios para evaluar la accesibilidad de la interfaz web**
ISO/IEC/IEEE 29119-3:2021	**Documentación de escenarios y procedimientos de prueba**
IEEE 1044-2009	**Clasificación y registro de defectos de software**

Estas referencias se aplicarán según el alcance de cada actividad y las evidencias disponibles. Su inclusión en el plan no significa que el sistema haya sido certificado formalmente bajo dichas normas, pero se han tomado como referencia para poder cumplir como norma en nuestro sistema.

Estrategia de aseguramiento de calidad

Las actividades de calidad se desarrollarán de forma progresiva, desde la revisión de los requisitos hasta las pruebas finales.

Fase	Actividad	Criterio de revisión
Requisitos	Revisar los requisitos funcionales y no funcionales	Claridad, coherencia y posibilidad de verificación
Diseño	Revisar el prototipo y sus recorridos	Navegación comprensible y consistencia visual
Desarrollo	Revisar el código disponible	Organización, validaciones y manejo de errores
Pruebas unitarias	Comprobar funciones individuales	Resultado correcto para entradas válidas e inválidas
Integración	Verificar la comunicación entre módulos	Respuestas y datos consistentes
Usabilidad	Revisar tareas principales	Facilidad para completar las tareas previstas
Accesibilidad	Evaluar los criterios WCAG aplicables	Contraste, interacción, etiquetas y estructura
Cierre	Consolidar resultados	Pruebas documentadas y defectos pendientes identificados

Las actividades se ejecutarán según la disponibilidad de cada módulo. Si una funcionalidad todavía no está implementada, se documentará como pendiente de verificación funcional, sin atribuirle resultados de pruebas inexistentes.

Diferencia entre QA y QC

El aseguramiento de calidad (QA) y el control de calidad (QC) son actividades complementarias, pero tienen propósitos diferentes.

Aspecto	QA — Aseguramiento de calidad	QC — Control de calidad
Enfoque	Procesos de desarrollo	Producto y resultados
Propósito	Prevenir defectos	Detectar defectos
Momento	Durante todo el ciclo de desarrollo	Durante revisiones y pruebas
Actividades	Definir estándares, procedimientos y revisiones	Ejecutar pruebas, inspeccionar pantallas y comprobar resultados
Ejemplo en FitControl	Establecer una lista de verificación para formularios	Comprobar si el login rechaza credenciales incorrectas
Evidencia	Planes, listas de revisión y registros de seguimiento	Casos ejecutados, resultados y reportes de defectos

En FitControl, QA permitirá organizar las actividades para prevenir problemas, mientras que QC permitirá comprobar el comportamiento del prototipo y de las funciones implementadas mediante revisiones y pruebas.

Criterios de calidad por módulo

Módulo	Criterio de aceptación
Login	Los campos deben validarse y los mensajes de error deben ser comprensibles
Dashboard	Las opciones de navegación deben ser identificables y dirigir a los destinos previstos
Clientes	Los formularios y operaciones implementadas deben validar los datos y mostrar resultados coherentes
Planes	La información de membresías debe presentarse de forma legible y consistente
Pagos	Cada método debe mostrar instrucciones claras; las operaciones implementadas deben informar el resultado real

Diseño adaptable	El contenido debe mantenerse utilizable en los tamaños de pantalla contemplados
Accesibilidad	Los criterios WCAG evaluados deben contar con resultados y evidencias documentadas

Enfoque Shift-Left

El enfoque Shift-Left consiste en incorporar actividades de calidad desde las primeras etapas del desarrollo, en lugar de esperar hasta que el sistema esté terminado.

En FitControl se aplicará de la siguiente manera:

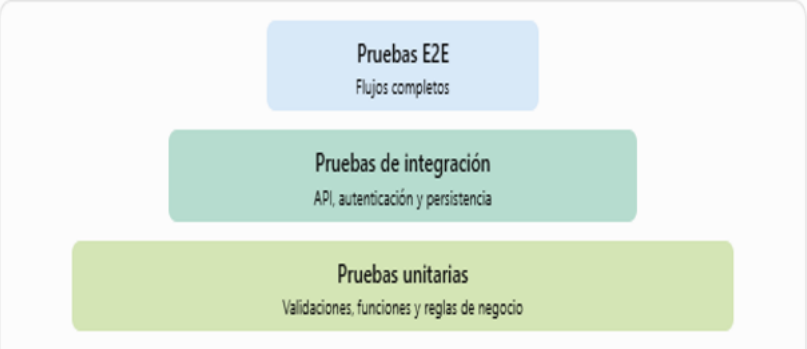
1. Análisis: revisar los requisitos antes de iniciar cada funcionalidad.
2. Diseño: detectar problemas de navegación y accesibilidad en el prototipo.
3. Desarrollo: revisar el código y las validaciones durante la implementación.
4. Integración: comprobar la comunicación entre módulos antes de incorporar nuevos cambios.
5. Pruebas: ejecutar casos desde que cada funcionalidad esté disponible.
6. Corrección: repetir las pruebas después de solucionar los defectos.

El beneficio esperado es detectar las desviaciones oportunamente y reducir el esfuerzo requerido para corregirlas en etapas posteriores, encontrando defectos o errores a tiempo para evitar que se extienda durante el desarrollo del sistema.

- **6.2 Pirámide de Pruebas Automatizadas (Unitarias, Integración, E2E)**

Pirámide de pruebas

La estrategia de pruebas de FitControl se organizará en tres niveles: pruebas unitarias, pruebas de integración y pruebas de extremo a extremo (E2E).



Pruebas unitarias

Verifican el comportamiento de unidades pequeñas del software de manera aislada.

En FitControl:

- Validación del formato del correo electrónico.
- Validación de campos obligatorios.
- Comprobación de reglas de cálculo, si existen en la implementación.
- Validación de datos de entrada en los servicios.

Pruebas de integración

Comprueban la interacción entre componentes.

En FitControl:

- Verificar la respuesta del servicio de autenticación.
- Comprobar la comunicación entre la interfaz y el backend.
- Verificar que una operación utilice correctamente la persistencia.
- Comprobar el tratamiento de errores de la API.

Pruebas de extremo a extremo (E2E)

Comprueban un recorrido completo desde la perspectiva del usuario.

En FitControl:

- Iniciar sesión y acceder al dashboard.
- Navegar desde el dashboard hasta el módulo de clientes.
- Seleccionar un plan y avanzar al proceso de pago.
- Completar un flujo de pago y comprobar el resultado mostrado.

Las pruebas E2E deberán ejecutarse únicamente sobre recorridos que estén implementados. Los recorridos disponibles exclusivamente en Figma se documentarán como pruebas de navegación del prototipo, no como pruebas E2E del software.

6.3 Clasificación y Flujo de Gestión de Defectos (IEEE 1044-2009)

Clasificación de defectos

Los errores encontrados deberán de clasificarse siguiendo los siguientes criterios, ya que con esto permitirá tomar decisiones para la resolución de los defectos que se puedan encontrar evaluando la prioridad que se tenga:

Funcional	Un formulario no ejecuta la acción esperada
Visual	Un elemento se superpone con otro
Usabilidad	El usuario no identifica cómo regresar a la pantalla anterior
Accesibilidad	El contraste de un texto resulta insuficiente
Integración	La interfaz no procesa correctamente una respuesta de la API
Datos	Se muestran o guardan datos incorrectos
Documentación	Las instrucciones de ejecución están incompletas

Severidad y prioridad

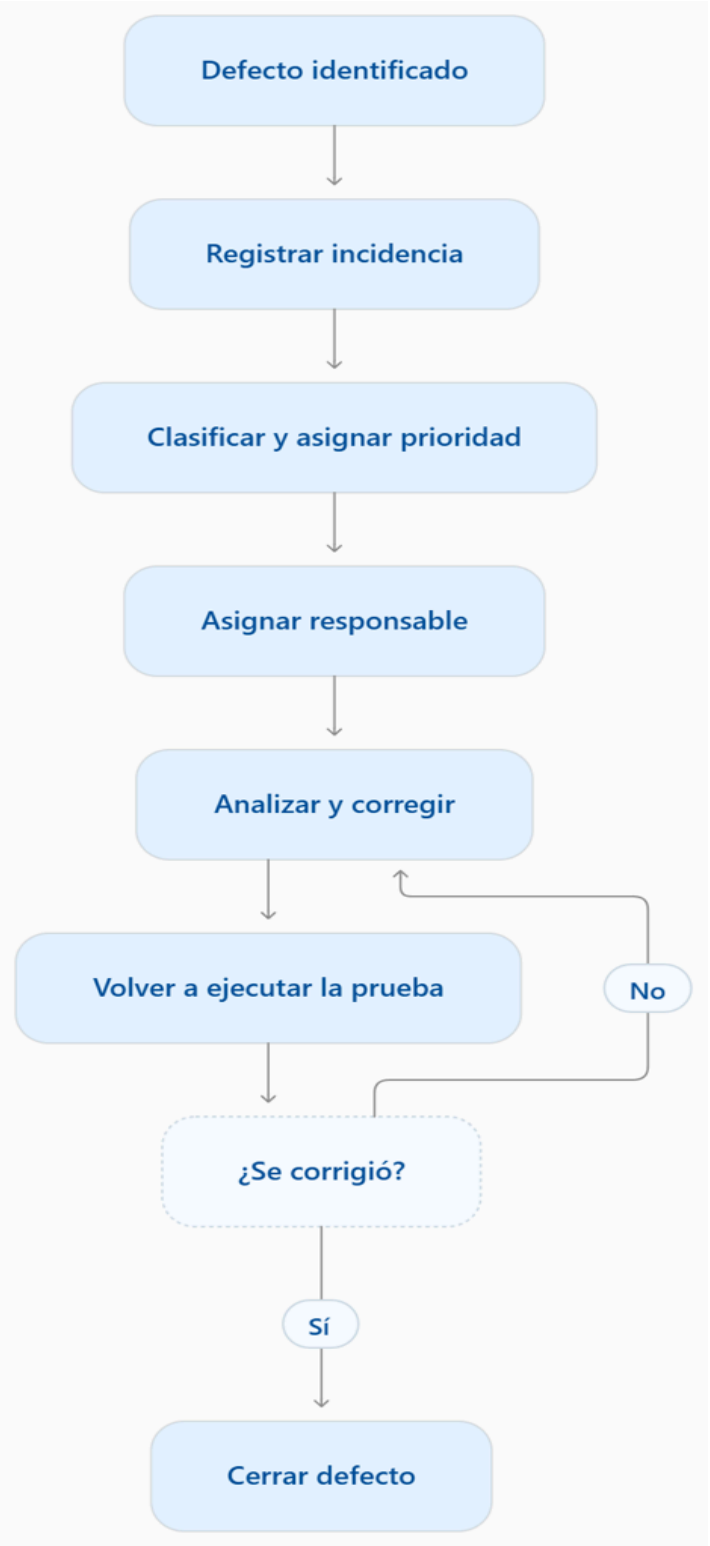
Severidad: expresa el impacto del defecto.

Prioridad: indica qué tan pronto debe atenderse.

Nivel	Severidad	Prioridad
Crítica	Impide una función esencial o compromete gravemente la seguridad	P1: atención inmediata
Alta	Bloquea una función importante sin alternativa adecuada	P2: corrección urgente
Media	Afecta parcialmente una función, con alternativa disponible	P3: corrección planificada
Baja	Problema menor visual o documental	P4: corrección cuando corresponda

La clasificación debe basarse en el impacto real, no únicamente en la apariencia del problema.

Ciclo de vida del defecto:



Plantilla para registrar defectos

Para que se registre un defecto se debe de registrar con la siguiente plantilla para que esta sea tomada en cuenta, ya que se busca que se tenga la información necesaria para poder evaluarla y resolverla sin tener que realizar un proceso largo de investigación para solucionar el defecto.

Campo	Contenido
ID	**DEF-001, DEF-002, etc.**
Fecha	**Día en que se detectó**
Módulo	**Login, clientes, planes, pagos, etc.**
Descripción	**Qué ocurrió y qué se esperaba?**
Pasos para reproducir	**Acciones necesarias para repetir el problema**
Resultado obtenido	**Comportamiento observado**
Severidad	**Crítica, alta, media o baja**
Prioridad	**P1, P2, P3 o P4**
Evidencia	**Captura, registro o respuesta de error**
Responsable	**Integrante asignado**
Estado	**Abierto, en corrección, en verificación o cerrado**
Resolución	**Cambio realizado y resultado de la nueva prueba**

Vínculos:

https://github.com/J0seph2000/modulo-membresia/blob/main/Entregable%20Final/proyecto/docs/qa/plan_sqa_ieee730.pdf

https://github.com/J0seph2000/modulo-membresia/blob/main/Entregable%20Final/proyecto/docs/qa/reporte_defectos_ieee1044.pdf

7. Matriz de trazabilidad integral

Requerimientos ↔ Contratos API (OpenAPI) ↔ Escenarios BDD ↔ Casos de prueba

1. Catálogo de requisitos

id	Tipo	Descripcion	Prioridad
RF-01	Funcional	La API debe informar su estado de salud.	Alta
RF-02	Funcional	Registrar usuarios con validación de datos (usuario, password ≥ 6, rol), contraseña con hash bcrypt y rechazo de duplicados.	Alta
RF-03	Funcional	Iniciar sesión y emitir un token JWT; rechazar credenciales incorrectas o incompletas.	Alta
RF-04	Seguridad	Proteger los endpoints de negocio: sin token válido responder 401 con WWW-Authenticate.	Alta
RF-05	Funcional	Gestionar clientes (consultar, crear, actualizar, eliminar) con validación e integridad referencial.	Alta
RF-06	Funcional	Consultar y crear planes de membresía con validación de nombre y precio.	Media
RF-07	Funcional	Registrar y consultar membresías (fechas válidas, cliente y plan existentes).	Media
RF-08	Funcional	Filtrar la lista de clientes por estado y por texto (nombre/apellido).	Media
RF-09	No funcional (rendimiento)	Cachear la consulta del catálogo de planes (X-Cache HIT/MISS) e invalidarla al crear un plan.	Media
RNF-01	No funcional (interoperabilidad)	Todas las respuestas de error usan application/problem+json (RFC 9457) con códigos HTTP según RFC 9110.	Alta
RNF-02	No funcional (API-First)	El contrato OpenAPI 3.0.3 describe todos los endpoints, esquemas, parámetros y ejemplos.	Alta

2. Matriz de trazabilidad (por requisito)

Cada requisito está vinculado a los endpoints del contrato docs/api/openapi.yaml (operationId), a los escenarios Gherkin de features/ que lo verifican, a sus casos de prueba y a la verificación automatizada adicional.

Requisito	Contrato API (operationId — ruta)	Escenarios BDD (feature · escenario)	Casos de prueba	Verificación automatizada
RF-01 Funcional	health — GET /api/health	clientes · La API informa que está activa	TC-01	tests/bdd + Postman
RF-02 Funcional	registrarUsuario — POST /api/auth/register	autenticacion · Registro exitoso de un usuario nuevo autenticacion · Registro rechazado por datos inválidos autenticacion · Registro rechazado porque el usuario ya existe	TC-02, TC-03, TC-04	tests/unit/authService.test.js

RF-03 Funcional	iniciarSesion — POST /api/auth/login	autenticacion · Inicio de sesión exitoso devuelve un token JWT autenticacion · Inicio de sesión rechazado	TC-05, TC-06	tests/unit/authService.test.js
RF-04 Seguridad	listarClientes — GET /api/clientes listarPlanes — GET /api/planes listarMembresias — GET /api/membresias	autenticacion · Los endpoints protegidos rechazan peticiones sin credenciales válidas	TC-07	tests/unit/errorHandler.test.js
RF-05 Funcional	listarClientes — GET /api/clientes crearCliente — POST /api/clientes obtenerCliente — GET /api/clientes/{id} actualizarCliente — PUT /api/clientes/{id} eliminarCliente — DELETE /api/clientes/{id}	clientes · Listar los clientes clientes · Crear un cliente válido clientes · Crear un cliente con datos inválidos clientes · Consultar un cliente por un id inexistente o inválido clientes · Actualizar un cliente existente clientes · Eliminar un cliente existente clientes · Filtrar clientes por texto clientes · Filtrar clientes por estado clientes · El filtro por estado rechaza valores desconocidos membresias · No se puede eliminar un cliente que tiene membresías	TC-08, TC-09, TC-10, TC-11, TC-12, TC-13, TC-14, TC-15, TC-16, TC-25	tests/unit/clienteService.test.js
RF-06 Funcional	listarPlanes — GET /api/planes crearPlan — POST /api/planes	planes · Listar los planes planes · Crear un plan válido planes · Crear un plan con datos inválidos planes · La consulta de planes usa caché y se invalida al crear un plan	TC-17, TC-18, TC-19, TC-20	tests/unit/planService.test.js
RF-07 Funcional	listarMembresias — GET /api/membresias crearMembresia — POST /api/membresias eliminarCliente — DELETE /api/clientes/{id}	membresias · Listar las membresías membresias · Crear una membresía válida membresias · Crear una membresía con fechas inválidas membresias · Crear una membresía con un cliente inexistente membresias · No se puede eliminar un cliente que tiene membresías	TC-21, TC-22, TC-23, TC-24, TC-25	tests/unit/membresiaService.test.js

RF-08 Funcional	listarClientes — GET /api/clientes	clientes · Filtrar clientes por texto clientes · Filtrar clientes por estado clientes · El filtro por estado rechaza valores desconocidos	TC-14, TC-15, TC-16	tests/unit/clienteService.test.js
RF-09 No funcional (rendimiento)	listarPlanes — GET /api/planes crearPlan — POST /api/planes	planes · La consulta de planes usa caché y se invalida al crear un plan	TC-20	tests/unit/planService.test.js
RNF-01 No funcional (interoperabilidad)	Componente Problem / ProblemValidation y respuestas 4xx/5xx de cada operación	autenticacion · Registro rechazado por datos inválidos autenticacion · Registro rechazado porque el usuario ya existe autenticacion · Inicio de sesión rechazado … y 8 escenarios más (ver sección 3)	TC-03, TC-04, TC-06, TC-07, TC-10, TC-11, TC-16, TC-19, TC-23, TC-24, TC-25	tests/unit/errorHandler.test.js
RNF-02 No funcional (API-First)	Los 12 operationId del contrato (ver sección 3)	clientes · La API informa que está activa autenticacion · Registro exitoso de un usuario nuevo autenticacion · Registro rechazado por datos inválidos … y 22 escenarios más (ver sección 3)	TC-01, TC-02, TC-03, TC-04 … (25 casos)	docs/api/openapi.yaml + /api-docs

3. Matriz inversa (caso de prueba → requisito → contrato)

TC-01	RF-01, RNF-02	health — GET /api/health	clientes · La API informa que está activa	Escenario
TC-02	RF-02, RNF-02	registrarUsuario — POST /api/auth/register	autenticacion · Registro exitoso de un usuario nuevo	Escenario
TC-03	RF-02, RNF-01, RNF-02	registrarUsuario — POST /api/auth/register	autenticacion · Registro rechazado por datos inválidos	Esquema (4 ej.)
TC-04	RF-02, RNF-01, RNF-02	registrarUsuario — POST /api/auth/register	autenticacion · Registro rechazado porque el usuario ya existe	Escenario
TC-05	RF-03, RNF-02	iniciarSesion — POST /api/auth/login	autenticacion · Inicio de sesión exitoso devuelve un token JWT	Escenario
TC-06	RF-03, RNF-01, RNF-02	iniciarSesion — POST /api/auth/login	autenticacion · Inicio de sesión rechazado	Esquema (3 ej.)
TC-07	RF-04, RNF-01, RNF-02	listarClientes — GET /api/clientes listarPlanes — GET /api/planes listarMembresias — GET /api/membresias	autenticacion · Los endpoints protegidos rechazan peticiones sin credenciales válidas	Esquema (5 ej.)
TC-08	RF-05, RNF-02	listarClientes — GET /api/clientes	clientes · Listar los clientes	Escenario
TC-09	RF-05, RNF-02	crearCliente — POST /api/clientes	clientes · Crear un cliente válido	Escenario
TC-10	RF-05, RNF-01, RNF-02	crearCliente — POST /api/clientes	clientes · Crear un cliente con datos inválidos	Esquema (3 ej.)
TC-11	RF-05, RNF-01, RNF-02	obtenerCliente — GET /api/clientes/{id}	clientes · Consultar un cliente por un id inexistente o inválido	Esquema (4 ej.)

TC-12	RF-05, RNF-02	actualizarCliente — PUT /api/clientes/{id}	clientes · Actualizar un cliente existente	Escenario
TC-13	RF-05, RNF-02	eliminarCliente — DELETE /api/clientes/{id} obtenerCliente — GET /api/clientes/{id}	clientes · Eliminar un cliente existente	Escenario
TC-14	RF-05, RF-08, RNF-02	listarClientes — GET /api/clientes	clientes · Filtrar clientes por texto	Escenario
TC-15	RF-05, RF-08, RNF-02	listarClientes — GET /api/clientes	clientes · Filtrar clientes por estado	Esquema (2 ej.)
TC-16	RF-05, RF-08, RNF-01, RNF-02	listarClientes — GET /api/clientes	clientes · El filtro por estado rechaza valores desconocidos	Escenario
TC-17	RF-06, RNF-02	listarPlanes — GET /api/planes	planes · Listar los planes	Escenario
TC-18	RF-06, RNF-02	crearPlan — POST /api/planes	planes · Crear un plan válido	Escenario
TC-19	RF-06, RNF-01, RNF-02	crearPlan — POST /api/planes	planes · Crear un plan con datos inválidos	Esquema (3 ej.)
TC-20	RF-06, RF-09, RNF-02	listarPlanes — GET /api/planes crearPlan — POST /api/planes	planes · La consulta de planes usa caché y se invalida al crear un plan	Escenario
TC-21	RF-07, RNF-02	listarMembresias — GET /api/membresias	membresias · Listar las membresías	Escenario
TC-22	RF-07, RNF-02	crearMembresia — POST /api/membresias	membresias · Crear una membresía válida	Escenario
TC-23	RF-07, RNF-01, RNF-02	crearMembresia — POST /api/membresias	membresias · Crear una membresía con fechas inválidas	Esquema (3 ej.)
TC-24	RF-07, RNF-01, RNF-02	crearMembresia — POST /api/membresias	membresias · Crear una membresía con un cliente inexistente	Escenario
TC-25	RF-05, RF-07, RNF-01, RNF-02	eliminarCliente — DELETE /api/clientes/{id}	membresias · No se puede eliminar un cliente que tiene membresías	Escenario

4. Resumen de cobertura

Requisitos totales: 11 · Requisitos con al menos un caso de prueba: 11 · Casos de prueba: 25 · Operaciones del contrato cubiertas: 12 de 12.

Requisitos sin cobertura: ninguno.

8. CONCLUSIONES Y LECCIONES APRENDIDAS

Durante el desarrollo del Módulo de Membresías se comprendió que documentar un sistema de software no es un trámite, sino una herramienta de trabajo. Definir con precisión los requerimientos, los componentes y sus interfaces obligó al equipo a tomar decisiones de diseño que de otra forma habrían quedado pendientes hasta la implementación. El resultado es un módulo con alcance claro, arquitectura justificada y requerimientos trazables de principio a fin.

Lecciones aprendidas

1. La trazabilidad debe mantenerse desde el inicio. Esperar a terminar todos los documentos para cruzar requerimientos, componentes y diagramas generó retrabajo. Actualizar la matriz de trazabilidad en paralelo con cada entregable habría ahorrado tiempo y evitado inconsistencias.
2. Los casos de error son tan importantes como el flujo principal. Al redactar las fichas de componentes IEEE 1016 se descubrieron escenarios que no estaban en el SRS, como el comportamiento ante un timeout de la pasarela de pagos. Pensar en los errores desde la especificación de requerimientos produce un diseño más robusto.